GENERATIVE AI-ASSISTED SOFTWARE DEVELOPMENT TEAMS: OPPORTUNITIES, CHALLENGES, AND BEST PRACTICES

Lappeenranta–Lahti University of Technology LUT

Degree Programme in Software Engineering, Master's thesis

2025

Nouman Noor

Examiners: Professor Kari Smolander

Abstract

Lappeenranta–Lahti University of Technology LUT

LUT School of Engineering Sciences

Digital Systems and Service Development

Nouman Noor

Generative AI-Assisted Software Development Teams: Opportunities, Challenges, and Best Practices

Master's Thesis

2025

84 pages, 1 figures, 6 tables and 1 appendix

Examiners: Professor Kari Smolander

Keywords: Generative AI, Software development, Software Teams, Copilot, ChatGPT, Startups

This thesis investigates the integration of generative artificial intelligence (AI) tools into software development teams, with a specific focus on startup environments. It examines how AI-powered tools like GitHub Copilot and ChatGPT affect team productivity, workflows, skill development, and risk management. The study follows a qualitative research approach using semi-structured interviews with eight professionals working in AI-assisted development roles.

The findings indicate that generative AI can enhance productivity, automate routine tasks, assist in debugging and documentation, and support continuous learning. However, challenges include AI hallucinations, tool-context mismatch, data privacy risks, and skill dependency. Thematic analysis reveals both technical and organizational considerations, and the thesis proposes best practice guidelines for startups: starting with low-risk tasks, maintaining human oversight, developing prompt literacy, and defining AI usage boundaries.

This study contributes actionable insights for teams seeking to integrate AI into their development lifecycle while preserving software quality, team autonomy, and long-term innovation capabilities.

Acknowledgements

This journey began in Panjpir, a small town in northern Pakistan, and has taken me all the way to LUT University in Finland. Along the way, I've been fortunate to grow both personally and academically, thanks to the unwavering support and kindness of many people.

I would like to express my gratitude to Professor Kari Smolander, my supervisor, for his consistent guidance and insightful feedback throughout this thesis. His support made this work possible.

I am sincerely thankful to the software professionals from various startups who generously participated in this study. Their openness in sharing their experiences made this research meaningful.

To the faculty and staff at LUT University, especially in the Digital Systems and Service Development program,thank you for providing a supportive academic environment.

Above all, I want to thank the people closest to me, my family, whose love and sacrifices have carried me through. To my father, Noor Khan, and my mother, thank you for believing in me, even when the road ahead seemed uncertain. To my uncle, Zahid Ali, and my elder brother, Faizan Ali, thank you for your unwavering support, encouragement, and wisdom. I also extend my heartfelt gratitude to my brothers, Azan and Sohaib, and my sisters, Nida and Farhana, whose love and encouragement have been a constant source of strength. This achievement is as much yours as it is mine.

From the streets of Panjpir to the lecture halls of LUT, this thesis is a reflection of all the people who have helped me along the way. I dedicate this work to you, and with all my heart, I thank you.

List of Symbols and Abbreviations

Abbreviation / Symbol	**Meaning**
AI	Artificial Intelligence
LLM	Large Language Model
IDE	Integrated Development Environment
GPT	Generative Pre-trained Transformer (e.g., GPT-3.5, GPT-4)
IP	Intellectual Property
QA	Quality Assurance
API	Application Programming Interface
SDLC	Software Development Life Cycle
SME	Small and Medium-sized Enterprise
P1–P8	Anonymized participant identifiers used in interviews
Copilot	GitHub Copilot – AI-powered code completion assistant
ChatGPT	Conversational AI tool developed by OpenAI
Tabnine	AI-based code completion tool focused on privacy
DeepCode	AI-powered code review and static analysis tool (Snyk Code)
NVivo	Qualitative data analysis software used for coding and theme development
Prompt Literacy	The skill of crafting effective prompts to interact with AI tools

Table of contents

Abstract

Acknowledgements

List of Symbols and Abbreviations 4

Table of contents 5

Lists of figures and tables (optional) 10

1 Introduction 11

1.1 Background and Context 11

1.2 Generative AI in Software Development 12

1.2.1 Code generation and completion: 12

1.2.2 Automated documentation and explanation: 12

1.2.3 Testing and quality assurance: 13

1.2.4 DevOps and CI/CD automation: 13

1.3 Problem Statement 13

1.4 Research Objectives 15

1.5 Research Questions 15

1.6 Significance of the Study 16

1.6.1 Academic Significance 16

1.6.2 Practical Significance 16

1.7 Structure of the Thesis 17

1.7.1 Chapter 2: Literature Review 17

1.7.2 Chapter 3: Research Methodology 17

1.7.3 Chapter 4: Results 17

1.7.4 Chapter 5: Discussion 18

1.7.5 Chapter 6: Conclusion 18

2 Literature Review 19

2.1 Capabilities of Generative AI in Coding 19

2.1.1 Code Generation 19

2.1.2 Automated Debugging and Testing 20
2.1.3 Documentation and Maintenance 21
2.2 AI Integration in Startup Environments 22
2.3 Challenges of AI-Assisted Coding 23
2.3.1 Over-Reliance and Skill Decay 24
2.3.2 Security and IP Risks 24
2.3.3 Tool Reliability and Accuracy 25
2.3.4 Ethical Concerns and Bias 26
2.4 Gaps in Current Research 27
3 Research Methodology 29
3.1 Research Design and Rationale 29
3.2 Data Collection: Semi-Structured Interviews 29
3.3 Participant Selection 30
3.3.1 Recruitment 30
3.3.2 Participant Details 31
3.4 Data Analysis Method: Thematic Analysis 31
3.4.1 Braun and Clarke's Six Phases of Thematic Analysis 31
3.4.2 Thematic Analysis Process and Implementation 32
3.5 Ethical Considerations 34
3.5.1 Informed Consent 35
3.5.2 Anonymity and Confidentiality 35
3.5.3 Participant Review 35
3.5.4 Data Protection and Storage 35
3.6 Validity and Limitations 36
3.6.1 Credibility 36
3.6.2 Transferability 36
3.6.3 Dependability 36
3.6.4 Confirmability 37
3.6.5 Limitations 37

4 Findings and Thematic Analysis 38
4.1 Overview of Participants 38
4.2 Summary of Coding and Theme Development 38
4.3 Benefits of Generative AI in Startups 40
4.3.1 Refactoring with AI Tools 40
4.3.2 Enhancing User Story Clarity 42
4.3.3 Automating Daily Standups 42
4.3.4 AI-Supported Testing and Debugging 43
4.3.5 Cross-Team Documentation Sharing 44
4.3.6 AI as a Tool for Learning and Skill Development 45
4.3.7 Strategic Time Reallocation 46
4.3.8 Empowerment of Non-Technical Staff 47
4.4 Challenges of AI in Startups 49
4.4.1 Tool–Project Mismatch 49
4.4.2 Hallucinated Functions and Suggestions 50
4.4.3 Data Privacy Concerns 51
4.4.4 Over-Reliance and Dependency 52
4.4.5 Reduced Peer Interaction 53
4.4.6 Prompt Skill Requirement 54
4.4.7 Cultural Resistance 55
4.5 Summary of Tool Usage 55
5 Discussion 57
5.1 Reflection on Key Findings 57
5.1.1 Improved Productivity and Efficiency 57
5.1.2 Support for Documentation and Knowledge Sharing 57
5.1.3 Trust and Reliance on AI 58
5.1.4 Shifts in Collaboration and Team Dynamics 59
5.2 Alignment with Previous Research 59
5.2.1 Productivity Gains and Modes of AI Use 60

5.2.2 Documentation and Communication 61
5.2.3 Reliability Concerns and Trust 61
5.2.4 Impacts on Team Collaboration and Roles 62
5.3 Implications for Practice 63
5.3.1 Evolving Developer Roles and Skills 63
5.3.2 Changes in Team Workflow 64
5.3.3 Project Management and Planning 64
5.3.4 Empowering QA and DevOps 65
5.4 Challenges in Real-World Adoption 65
5.4.1 Maintaining Trust in AI Outputs 65
5.4.2 Hallucinations and Reliability of AI 66
5.4.3 Tool Appropriateness and Integration (Tool Fit) 66
5.4.4 Prompt Skill Requirements and Team Training 67
5.4.5 Resistance and Ethical Concerns 67
5.5 Recommendations for Startups 68
5.5.1 Start with Low-Risk Tasks 69
5.5.2 Avoid Using AI with Sensitive or Proprietary Data 70
5.5.3 Develop Prompt Literacy Across the Team 70
5.5.4 Use AI to Support, Not Replace, Critical Thinking 71
5.5.5 Empower Non-Technical Roles with AI 72
5.5.6 Define Clear Boundaries for AI Use 73
5.5.7 Maintain Human Oversight on Core Logic and Architecture 74
5.6 Limitations and Suggestions for Future Research 75
6 Conclusion 78
6.1 Summary of Key Findings 78
6.2 Contributions to Practice 79
6.3 Final Thoughts 80
References 82
Appendices

Appendix Interview Questions for Software Developers

List of Figures

1 Schematic of Braun & Clarke's six-phase thematic analysis process, as implemented in this study.

List of Tables

1 Anonymized Participant Demographics
2 Example of Codes and Corresponding Themes (synthesized sample)
3 Development of Benefit Themes from Codes
4 Development of Challenge Themes from Codes
5 AI Tools Used, Functions, and Participant Use Cases
6 Guidelines for AI Adoption in Startup Development Teams

1 Introduction

1.1 Background and Context

In recent years, generative artificial intelligence (AI) has rapidly emerged as a transformative force in software engineering. The release of advanced large language models like OpenAI's GPT-4 (2023) and the introduction of code-focused AI systems such as OpenAI Codex (which powers GitHub Copilot) have enabled AI to produce and assist in writing software code at an unprecedented level (Gupta et al., 2025). These generative AI tools can understand natural language prompts and generate functional code, effectively acting as AI pair programmers that can collaborate with human developers. As a result, software development practices are beginning to change: early studies report that developers using AI coding assistants like Copilot can complete programming tasks significantly faster (over 50% faster on average for certain languages) and with less effort (Gupta et al., 2025).

This paradigm shift has not gone unnoticed by industry. A recent Gartner survey found that two-thirds of businesses are already piloting or deploying AI-driven coding tools, and it is projected that by 2028 about 75% of engineering professionals will be utilizing AI coding assistants to boost development efficiency (Gupta et al., 2025). A recent developer survey shows that there is near universal interest in these tools – more than 97% of software developers in a 2024 multicountry survey claimed they had used AI coding tools at least once during the course of their work. (Daigle & Staff, 2024). In no time, generative AI has progressed from being a curious innovation to widely available in the software industry. For rapid-developing small software companies and agile teams facing strict resource limitations, these technologies are desirable options for speeding up coding, automating repetitive tasks, and enhancing the skills of the team.

Recent research indicates that AI can function not just as a tool, but as a 'cybernetic teammate' for human developers. It allows individuals to match the output of entire teams. In agile startup environments where speed and innovation are primary importance, being "AI-augmented" is increasingly seen as preferable to falling behind competitors who leverage these advancements (Wolpers, 2025).

These technologies also introduce new complexities to software engineering work. Integrating AI assistants into existing team workflows may disrupt established development practices and team dynamics. Questions are being raised about how relying on AI for coding might affect code quality, developer learning, and collaboration within teams. Early adopters have noted both significant benefits and emerging challenges. For example, while AI assistants can boost productivity, their suggestions are not error-free and may introduce errors or secu-

rity vulnerabilities that developers need to catch (Gupta et al., 2025).

1.2 Generative AI in Software Development

Generative AI in the context of software development refers to AI systems that can produce software artifacts (such as code, documentation, or test cases) in response to inputs or prompts, often using large language models trained on vast repositories of code. These tools use machine learning to generate new content that resembles what a human developer might create, effectively automating or assisting various programming tasks (Banh, Holldack & Strobel, 2025).

Generative AI tools like OpenAI's ChatGPT and GitHub Copilot have demonstrated "unique content creation capabilities" that offer transformative potential in how software is built and maintained (Banh, Holldack & Strobel, 2025). In practice, this means developers can describe a functionality in natural language and receive suggested code implementations. They can also ask for an explanation of a code snippet and get an AI-generated description. Generative AI thus acts as an intelligent assistant throughout the software development lifecycle.

1.2.1 Code generation and completion:

The most prominent use of generative AI is the automatic generation of source code. AI coding assistants when given a task description can produce code snippets, complete function bodies, or suggest entire algorithms to implement the required functionality (Gupta et al., 2025). For example, GitHub Copilot can take a comment like "// function to sort a list of strings by length" and generate a plausible implementation in the target programming language. This capability accelerates coding by handling boilerplate and routine code patterns. With these improvements developers have more time to focus on higher-level problem solving. Surveys show that a large majority of developers currently using AI tools do so primarily to assist in writing code, highlighting code generation as the leading use case for generative AI in practice (Gupta et al., 2025). Generative AI can significantly reduce development time and effort, although developers still need to review and debug the AI-produced code for correctness and quality (Gupta et al., 2025).

1.2.2 Automated documentation and explanation:

Generative AI is also increasingly used to support software documentation and knowledge sharing. This includes generating documentation comments, user manuals, or summaries of code behavior in natural language. AI tools can read the source code and produce human-readable explanations or even diagrams, which is valuable for onboarding team members or clarifying complex logic. In agile teams where documentation is often minimal, an AI

assistant can fill the gap by automatically creating up-to-date docstrings or README content based on code changes. Developer interest in this capability is high: In one survey, 81% of developers anticipated a greater use of AI to document code in the near future (Gupta et al., 2025).

1.2.3 Testing and quality assurance:

AI tools can help generate unit tests, integration test cases, or even find potential bugs by analyzing code for error patterns. For example, generative AI can produce a suite of unit test functions for a given module, covering various scenarios, which developers can then refine (Gupta et al., 2025). Some AI-based tools also assist in dynamic testing by generating inputs that might trigger edge cases or by suggesting fixes for identified bugs. In fact, automated test generation is one of the top areas where developers are curious to leverage AI (Gupta et al., 2025). By catching issues early through AI-generated tests, teams can improve software reliability and free up developers' time from writing tedious test code. AI-generated tests and code still require human check. Developers must validate that the tests are correct and meaningful, and ensure that the AI has not missed context-specific edge cases (Gupta et al., 2025).

1.2.4 DevOps and CI/CD automation:

AI can assist in writing configuration files (e.g. CI pipelines, deployment scripts), optimizing cloud resource configurations, or even automating routine operational tasks. For example, an AI agent might automatically generate a Dockerfile or a Kubernetes configuration based on a high-level description of an application's requirements. This use of AI in the software delivery pipeline is still emerging, but it is assured to grow rapidly. Market analyses predict explosive growth in AI-driven DevOps tools; one report projects the AI in DevOps market to grow from about $2.6 billion in 2022 to $10.4 billion by 2027, reflecting a compound annual growth rate of over 30% (Gupta et al., 2025).

1.3 Problem Statement

Despite the rapid advances in generative AI far less is formally known about what happens when an entire team in a real-world project setting integrates generative AI into their day-to-day work. In the context of large tech companies, some early studies and industry reports have begun to identify potential opportunities (e.g. productivity gains) as well as challenges (e.g. the need for oversight, changes in team communication) arising from AI-assisted development (Banh, Holldack & Strobel, 2025). Yet, in smaller organizations such as software startups – which often have fluid team roles, less formal process, and a need to rapidly iterate – these impacts are not well documented in the literature. Key questions remain unanswered:

For example, does having an 'AI pair programmer' change how junior and senior developers collaborate or share knowledge? Does reliance on AI-generated code affect the overall quality of the software product positively (by catching mistakes) or negatively (by introducing subtle bugs or technical debt)? How do agile processes like daily stand-ups, code reviews, or sprint planning accommodate contributions from an AI assistant? This thesis put forward that without a clear, research-based understanding of such issues, software startups risk either under-utilizing generative AI or misapplying it in ways that could undermine team performance. The central problem addressed here is the absence of a structured and empirical understanding of the integration of generative AI in software team settings. In particular, there is a knowledge gap in how generative AI tools alter team workflows (the sequence and nature of development tasks), collaboration (communication and coordination among team members), and development quality (the robustness and maintainability of the code produced) in the fast-paced, resource-constrained environment of startups. Addressing this problem is crucial: as generative AI tools become more prevalent, startup teams need guidance on how to harness them effectively without disrupting the human-centric collaboration that agile methods emphasize. This research is undertaken to fill that gap by investigating the experiences of software development teams adopting generative AI, thereby providing a structured understanding of its impact in startup contexts.

1.4 Research Objectives

In light of the problem outlined above, the objectives of this research are defined as follows:

- **Primary Objective:** To develop a structured understanding of how generative AI tools affect software development teams in startup environments, specifically in terms of team workflows, collaborative practices, and the quality of development outcomes. The study seeks to produce an evidence-based analysis of the role generative AI is playing in agile startup teams and how it influences their work and results.
- **Secondary Objectives:**
  - **Identify Key Opportunities:** To identify and analyze the key opportunities and benefits that generative AI offers to software development teams – for example, improvements in productivity, faster development cycles, enhanced creativity, or higher code quality – particularly as relevant to startup and small-team settings.
  - **Identify Key Challenges:** To uncover and examine the main challenges, obstacles, or risks that arise from integrating generative AI into team workflows, including issues related to team communication, trust in AI outputs, potential degradation of skills, or maintenance and quality concerns.
  - **Propose Best Practices:** To formulate a set of best practices and recommendations for software development teams and managers on effectively leveraging generative AI tools. This includes strategies to maximize the benefits (opportunities) while mitigating the challenges identified, ensuring that AI assistance is used in a way that complements human teamwork and aligns with agile principles.

Through these objectives, the research aims to comprehensively address the thesis topic of "Generative AI-Assisted Software Development Teams: Opportunities, Challenges, and Best Practices." The primary objective provides the overall goal, while the secondary objectives break down the inquiry into targeted outcomes that together will answer the needs of both academia and industry practitioners.

1.5 Research Questions

To achieve the research objectives and address the identified knowledge gap, this thesis is guided by the following two research questions:

1. **RQ1: How are software development teams in startup environments integrating generative AI tools into their agile workflows, and what opportunities and chal-**

lenges are emerging from this integration?

This question explores the actual usage patterns, benefits, and obstacles encountered in adopting generative AI (e.g., GitHub Copilot, ChatGPT) within real-world startup software teams.

2. **RQ2: What best practices can be identified to guide startup software teams in effectively adopting and managing generative AI tools to enhance collaboration and development outcomes?**

   This question focuses on developing actionable recommendations based on empirical findings.

1.6 Significance of the Study

1.6.1 Academic Significance

This research is important to the academic community because it tackles a rapidly emerging phenomenon at the intersection of software engineering and artificial intelligence, an area where formal study is only beginning to catch up with practice. Generative AI's entrance into software development represents a novel socio-technical change – developers and intelligent machines collaborating to build software – which raises new theoretical and empirical questions. By providing an in-depth investigation of generative AI use in team settings, this study contributes to filling the current gap in software engineering literature regarding AI-assisted development. It builds upon initial studies (such as Banh et al. 2025) that have started to outline the potential of GenAI and the challenges of integrating it into existing processes (Banh, Holldack & Strobel, 2025). The present thesis extends this knowledge by focusing on startup environments and by synthesizing findings into a structured framework of opportunities and challenges. In doing so, it offers a timely scholarly insight and a conceptual basis for understanding how AI changes team dynamics and project outcomes. The study's conclusions – including identified best practices – can inform future research. It effectively lays groundwork for subsequent empirical studies and theory development on human-AI collaboration in software projects, a field likely to grow as AI becomes more embedded in development work.

1.6.2 Practical Significance

For software startups and agile teams, the importance of this study is immediate and high. As highlighted by recent industry interviews, majority of software professionals expect generative AI to become a standard part of their work in the next few years (Capgemini Research Institute, 2024). Many organizations are already experimenting with tools like ChatGPT

and Copilot. Early adopters report benefits such as increased productivity (with measured improvements of 7–18% in software engineering output) and enhanced software quality (Capgemini Research Institute, 2024). But, at the same time, most organizations have not yet scaled these AI tools across all projects, and there is uncertainty about best implementation practices (Capgemini Research Institute, 2024). This study provides practical guidance to those navigating this uncertainty. The thesis offers value to team leaders, startup CTOs, and developers considering or currently using generative AI. For example, if the research finds that certain collaboration practices or coding standards help mitigate the introduction of bugs by AI, these insights can directly inform how teams adjust their processes. If specific challenges (like decreased peer communication or oversight difficulties) are identified, the recommended best practices will help teams proactively address them – ensuring that the introduction of AI assistants actually yields net positive outcomes. This study serves as a roadmap for software startups on the effective adoption of generative AI.

1.7 Structure of the Thesis

To address the research questions and objectives, the thesis is organized into six chapters, as outlined below:

1.7.1 Chapter 2: Literature Review

This chapter reviews relevant literature and background on generative AI in software engineering. It covers the evolution of AI-assisted programming, including prior studies on tools like code generators and intelligent pair programming. The chapter also examines theoretical frameworks related to technology adoption in teams and agile methodology principles, in order to identify the expected impacts of generative AI on team workflows and collaboration.

1.7.2 Chapter 3: Research Methodology

This chapter describes the research design and methods used to conduct the study. It details whether the study follows a qualitative, quantitative, or mixed-methods approach and justifies the choice.

1.7.3 Chapter 4: Results

In this chapter, the findings from the empirical research are presented. The chapter is structured around the research questions or themes that emerged from the data. It reports on what was discovered regarding how teams use generative AI (RQ1), the opportunities/benefits observed (RQ2), and the challenges encountered.

1.7.4 Chapter 5: Discussion

This chapter provides an in-depth discussion of the results, interpreting what they mean in the context of the research questions and the existing literature. Here, the thesis connects the findings back to the concepts and expectations outlined in the Literature Review (Chapter 2).

1.7.5 Chapter 6: Conclusion

The final chapter concludes the thesis by summarizing the key findings and contributions. It revisits the research objectives and questions, highlighting how each has been addressed.

Each chapter of the thesis builds upon the previous ones to provide a comprehensive examination of generative AI-assisted software development teams. The structured approach, which moves from introduction and background, through methodology and findings, to discussion and conclusion, ensures that the reader is first grounded in context, then shown the evidence, and finally presented with interpretations and recommendations.

2 Literature Review

2.1 Capabilities of Generative AI in Coding

Generative AI has emerged as a transformative force in software development, automating and augmenting numerous coding tasks. AI-driven coding tools – like large language models (LLMs) capable of understanding and generating code – have significantly enhanced developer productivity and code quality (Alenezi & Akour, 2025). . Platforms like OpenAI's ChatGPT and GitHub Copilot exemplify these advances, assisting in everything from writing new code to analyzing existing code, and even influencing the skill sets now expected of developers in modern teams (Alenezi & Akour, 2025). The capabilities of generative AI in coding span three major areas: code generation, automated debugging and testing, and documentation & maintenance. Each of these is discussed below, along with benefits and considerations reported in recent literature.

2.1.1 Code Generation

One of the most prominent applications of generative AI in software engineering is automatic code generation. Modern LLMs such as OpenAI Codex and GPT-4 can translate natural-language prompts or specifications into functional source code (Madupati, 2024). For example, Codex (the model behind Copilot) is trained on vast code repositories and can produce executable code from simple text descriptions, dramatically reducing the time needed to implement a given functionality (Madupati, 2024). GPT-4 has the ability to generate code from docstrings and even solve complex programming challenges beyond human-level performance. These models can handle front-end interface code, interact with frameworks (even LaTeX for documentation), reverse-engineer algorithms, and execute code to verify outputs. Now developers can increasingly rely on AI to draft routine code segments, automating boilerplate coding work.

A number of AI-powered tools have brought these generative capabilities into development environments. GitHub Copilot, for example, integrates Codex into popular editors (VS Code, JetBrains, etc.) to provide real-time code suggestions and autocompletions, functioning as an "AI pair programmer" alongside the human coder (Coutinho et al., 2024). Copilot can suggest the next line or entire functions based on context, significantly speeding up coding. Copilot X (powered by GPT-4) extends these features further – beyond improved autocomplete, it can assist with understanding existing code, suggest improvements in pull requests, and even help with scripting and command-line tasks.

Empirical studies are beginning to quantify the productivity gains from AI code generation.

Developers using GitHub Copilot have been shown to complete programming tasks significantly faster and with potentially fewer errors than those coding without AI support (Alenezi & Akour, 2025). In a large experiment with professional programmers, Copilot users finished tasks 55% faster on average, and their code passed more unit tests on the first try (Alenezi & Akour, 2025). An internal case study at McKinsey found that teams equipped with generative coding tools were able to double their development speed for certain modules, with modest improvements observed in code quality and maintainability as well (Alenezi & Akour, 2025). Even individuals with minimal programming background can leverage tools like ChatGPT to generate usable code; in one interview, a non-developer P7 remarked "now I can write code with ChatGPT, and it explains everything when I ask", shows how AI lowers the barrier to entry for coding by providing guidance and explanations alongside code output.

Literature also urges a critical perspective on code generation via AI. AI-generated code is not guaranteed to be perfect, and there are documented cases of mistakes or mismatches with the user's true intent in complex scenarios (Alenezi & Akour, 2025). Developers must remain review and test AI-written code, as over-reliance on machine-generated solutions can lead to inefficient implementations (Alenezi & Akour, 2025). McKinsey's analysis observed that the effectiveness of AI suggestions can vary widely with task complexity, and maintaining "sufficient expertise to leverage AI assistance responsibly" is essential (Alenezi & Akour, 2025). Researchers have likewise cautioned about "over-dependence on AI" and stress that human oversight is crucial to ensure reliability and ethical use of generative code (Madupati, 2024). In high-stakes software projects, a developer's judgment is needed to verify AI contributions – a theme echoed by Piller et al. (2024), who note that while AI's creative outputs can spark ideas, "higher accuracy and trust are essential for tasks requiring domain-specific expertise (Piller, Srour & Marion, 2024).".

2.1.2 Automated Debugging and Testing

AI-driven tools can analyze code to detect bugs or suggest fixes. It can catch such issues that might be overlooked in manual reviews (Madupati, 2024). For instance, OpenAI's Codex and related models can assist in identifying errors in existing code by interpreting error messages. As noted by Participant P7 in one of the interviews conducted for this study, "a common use-case reported by developers is to paste a compiler or runtime error into a conversational AI like ChatGPT, which then explains the root cause and proposes a solution." This predictive debugging capability can significantly shorten troubleshooting time. IBM's deployment of an AI-powered defect prediction tool (trained on a decade of past project data) led to a measurable drop in software bugs – post-release defects decreased by 20%, and defect detection during development improved by 15%, with teams reporting a 30% faster debugging cycle on average (Alenezi & Akour, 2025).

Generative AI is being used to automatically create and optimize test cases. Rather than writing tests manually for each new feature, developers can rely on AI to suggest relevant test scenarios based on the code changes made (Alenezi & Akour, 2025). For example, given a function or module, an AI tool can generate unit tests that cover expected behaviors and even edge cases, informed by patterns learned from countless other programs (Alenezi & Akour, 2025). As the codebase evolves, AI can continuously update the test suite by predicting which areas are likely to break – a form of predictive testing aligned with continuous integration practices (Alenezi & Akour, 2025). Techniques like model-based testing and symbolic execution have been employed in AI systems to exercise a wide range of execution paths automatically, uncovering corner-case bugs that might escape typical manual testing (Alenezi & Akour, 2025).

Participants in the interviews conducted for this study described AI as a helpful aide for troubleshooting and test generation. As Participant P6 remarked, "I used AI to write the test cases… it even told me what edge cases I missed.". Another participant shared that when a build failed, they could simply feed the error into ChatGPT and "it tells me what's wrong and how to fix it."

2.1.3 Documentation and Maintenance

Generative AI has shown promise in streamlining software documentation and ongoing maintenance tasks. Documentation is an essential part of software projects but it is often neglected because it can be tedious to produce. AI tools are now helping automate documentation processes by generating useful descriptions and explanations. Recent research explores how well LLMs like ChatGPT can tailor software documentation to different audiences; for example, one study applied ChatGPT-4 to project documents and found it could produce high-quality summaries geared toward non-technical managers versus developers, respectively (Marchesi et al., 2025). Managers reported gaining a better understanding of the project from the adapted documentation, demonstrating the potential of AI to act as a context-aware documentation assistant that adjusts the content based on readership (Marchesi et al., 2025).

In practical team settings, AI tools are already being used to generate and organize documentation in ways that enhance maintenance and knowledge sharing. Developers reported using ChatGPT and similar assistants to create clearer code comments, module descriptions, and even commit messages that summarize code changes. As Participants P4 and P8 noted during interviews, these tools help improve documentation quality and reduce the cognitive load of writing descriptive text manually. One developer noted that when they push new code, they ask an AI tool to "generate the commit message and a short summary," which

helps the whole team quickly get what changed without having to read the diffs line-by-line. Participants have described taking a piece of complex code they don't understand and asking AI to "break it down line by line," effectively getting a tutorial on that code's logic

AI tools can automatically identify deprecated libraries or outdated API usages in a project improving the code's maintainability and longevity (Alenezi & Akour, 2025). By scanning dependency graphs and version histories, an AI assistant might alert a team that a certain framework in use has known vulnerabilities and suggest an upgrade path – work that traditionally requires manual vigilance. In later stages of the software lifecycle, AI-driven predictive maintenance systems analyze runtime data (logs, performance metrics, user patterns) to foresee potential issues before they escalate (Alenezi & Akour, 2025). Such systems employ anomaly detection and pattern recognition to flag deviations that could indicate an impending failure, allowing teams to address infrastructure or code problems proactively (for example, warning that a certain service is likely to crash under high load, or that a memory leak is developing) (Alenezi & Akour, 2025).

2.2 AI Integration in Startup Environments

Startups operate under limited resources, high pressure to innovate rapidly, and the need to adapt quickly to market feedback (Marchesi et al., 2025). Generative AI tools present an enticing opportunity to assist such teams by automating routine tasks, but successful integration requires careful strategy. Recent studies note that how to effectively incorporate generative AI into agile startup workflows remains an open question (Marchesi et al., 2025).To address this, both researchers and practitioners have begun outlining practical guidelines for adoption in startup settings. Based on interviews conducted for this study, a core recommendation is to introduce AI gradually, starting with low-risk tasks that won't jeopardize critical systems. Early use on peripheral activities (e.g., code formatting, test-case generation) helps teams build trust in AI, understand its limitations, and avoid costly mistakes in vital product areas.

Another key adoption guideline that emerged from the interviews is the importance of developing "prompt literacy" across the team—in other words, training staff to craft clear, contextual prompts that yield useful outputs. Participants found that without well-formulated prompts, AI assistants often return irrelevant or incorrect suggestions.

Participants also emphasized the need for organizations to define clear boundaries for AI use and maintain human oversight over critical decisions. Establishing upfront which tasks AI will handle versus what remains strictly under human responsibility helps set realistic expectations. For example, some startups allow AI tools to generate boilerplate code or documentation, while retaining architectural design and core algorithm development within human

control. Ensuring that human experts review AI-generated outputs before implementation was repeatedly mentioned as a safeguard to maintain code quality and accountability.

Empirical evidence is emerging that generative AI assistants can boost developer productivity significantly. For instance, in one field experiment developers using an AI coding tool completed tasks around 55% faster on average than a control group without AI. Similarly, a deployment at a large software firm reported that teams finished certain coding tasks almost twice as fast after integrating generative AI into their workflow. Along with speed, there are indications of modest improvements in code quality – one study observed fewer bugs and higher rates of tests passing on the first try when AI was involved. These efficiency gains can free developers from menial boilerplate coding, allowing them to reallocate time to more complex, high-value activities (Coutinho et al., 2024). In a pilot case, software professionals noted that AI tools streamlined their workflow and optimized time, enabling them to focus more on creative problem-solving and critical design work instead of repetitive tasks (Coutinho et al., 2024).

AI tools can also be used by non-technical team members to contribute to software projects, blurring the traditional role boundaries. For example, a product manager or designer in a startup can employ generative AI to draft initial requirement documents, user stories, or test scenarios in natural language, which the development team can then refine. Participants have indeed recommended empowering non-developers to use AI for tasks like specifications writing, documentation, and project planning. This inclusive use of AI fosters better communication between technical and non-technical staff, as it provides a common tool to express and iterate on ideas. Research in fact envisions AI-driven development platforms that enable stakeholders from varied backgrounds to contribute, review, and refine ideas in real-time, making the software creation process more inclusive, collaborative, and transparent (Coutinho et al., 2024).

2.3 Challenges of AI-Assisted Coding

The integration of generative AI into coding workflows also brings a range of challenges and risks that startups must carefully consider. Early adopters have observed that along with boosts in efficiency come concerns about reliability, human skills, security, and ethics. Unlike traditional software tools, AI assistants introduce unpredictability and opacity into the development process. In the following subsections, key challenge areas are discussed: the human factors of over-reliance on AI and potential skill decay among developers; security and intellectual property risks; the reliability and accuracy limitations of AI-generated code; and ethical considerations, including bias and accountability.

2.3.1 Over-Reliance and Skill Decay

One major concern is the risk of developers becoming over-reliant on AI tools, to the detriment of their own coding skills and problem-solving abilities. If a team comes to depend on an AI assistant for every routine coding task or solution, their capacity to think through complex problems or debug issues independently may deteriorate over time. The literature has begun to flag this issue – for example, a recent review identifies over-dependence on AI as a key challenge accompanying the adoption of coding assistants. Over-reliance can lead to skill decay, where developers practice critical thinking and algorithmic skills less frequently and thus become less proficient in them. Even the makers of AI tools acknowledge the need to keep human programmers in the loop. Studies stress that maintaining human expertise and oversight is vital – AI should augment human developers, not replace their judgment. Without this balance, there is a danger of "lazy" acceptance of AI outputs: developers might implement suggestions without fully understanding them. This phenomenon is especially risky for less-experienced engineers. In one experiment on AI-assisted coding, researchers found that the AI could sometimes mislead novice developers – for instance, by proposing a quick fix that involved changing test cases rather than correctly modifying the application code, thus leaving underlying bugs unresolved (Marchesi et al., 2025). To reduce over-reliance, participants in this study advised that teams treat AI-generated code as a starting point or source of inspiration, while continuing to apply rigorous code reviews, testing, and critical thinking to all outputs produced by AI tools.

2.3.2 Security and IP Risks

Using generative AI in software development raises important security and intellectual property (IP) concerns, as highlighted by participants in this study. One common issue is the handling of sensitive data. Since many AI coding tools operate in the cloud (e.g., OpenAI's Codex or ChatGPT), any code or information entered in a prompt may be stored or accessed by the service provider. Startup teams interviewed repeatedly cautioned against pasting confidential code or customer data into such tools. There is a genuine concern that proprietary algorithms or personal data could be leaked.

To address this, some startups reported enforcing internal policies that require developers to sanitize or abstract prompts. For example, rather than inputting actual source code, developers are instructed to describe the problem in generalized terms. This will helps avoid exposing sensitive or proprietary details. . Alongside privacy, generative AI also poses intellectual property dilemmas. AI-generated code blurs the lines of ownership: if an AI writes a snippet of code, does the copyright belong to the user, the tool provider, or no one at all? Legal scholars note that current IP frameworks have not caught up with this scenario, creating uncertainty for organizations using AI development tools (Alenezi & Akour, 2025).

Security vulnerabilities are another critical risk. AI tools can also introduce new bugs or insecure code if their suggestions are taken at face value. An AI assistant has no true understanding of security best practices; it might generate code that "works" for the prompt but is poorly optimized or exposes an exploit. Industry surveys of security leaders reveal significant concern that AI-generated code could lead to vulnerabilities and incidents if deployed without scrutiny (O'Flaherty, 2025). Without human oversight, an AI might produce code that is syntactically correct but fails to sanitize inputs, uses deprecated encryption methods, or neglects other security measures. Startups integrating AI must therefore follow strict review processes: every AI-drafted code segment should be audited for quality and security just as one would review a junior developer's work (Alenezi & Akour, 2025).

2.3.3 Tool Reliability and Accuracy

Generative AI tools do not guarantee correct or even runnable code – they generate outputs based on patterns, which means the results can be plausible-looking but wrong. Developers have encountered many instances where an AI's solution needs substantial debugging or does not fulfill the intended requirements. In a pilot study on integrating AI into real development work, participants noted frequent difficulties in getting the desired outcomes from the AI, and they often had to manually fix errors or inconsistencies in the generated (Marchesi et al., 2025). Common issues include the AI misinterpreting what the developer asked for, producing functionally incorrect logic, or failing to handle edge cases. Such inaccuracies mean that adopting AI doesn't eliminate work so much as change its nature – developers must spend time validating and correcting AI outputs, or else risk defects slipping into the software.

Not all tasks are equally well handled by current AI assistants, and this variability is itself a reliability concern. Studies have found that an AI's effectiveness can vary greatly with the complexity and context of the task. For straightforward, self-contained problems (e.g. writing a standard sorting function or simple CRUD database operations), modern code generators are often quite reliable. However, for complex tasks that require deep understanding of the specific application domain or involve intricate logic, AI suggestions can be off-target. For example, GitHub Copilot's own evaluation noted cases where the tool's outputs were misaligned with the application's requirements in complex scenarios, forcing developers to fall back on manual coding. In other words, the AI might "solve" a problem that the developer didn't intend to solve, simply because it misunderstood the context. This unpredictability makes it risky to trust AI code without testing. Even when the AI's code runs, it may contain subtle bugs or inefficiencies that are not immediately obvious.

2.3.4 Ethical Concerns and Bias

The integration of AI assistants in software development also raises broader ethical issues, particularly around bias, fairness, and accountability. AI systems learn from data, and if that training data carries biases, the AI will likely reproduce or even amplify those biases in its outputs (Alenezi & Akour, 2025). In the coding domain, For instance, an AI tool trained predominantly on legacy systems or a narrow set of projects might favor certain programming patterns or libraries that reflect historical or majority preferences, potentially overlooking newer, more diverse solutions. As one study notes, a model might inadvertently underrepresent certain programming paradigms or technologies if those were less present in its training corpus (Alenezi & Akour, 2025). This could lead to recommendations that are not truly optimal or inclusive of alternative approaches, thereby skewing the decision-making process. In a team context, such bias might mean the AI often suggests solutions using a familiar tech stack even when an out-of-trend but suitable option exists, or it might prioritize requirements in a way that mirrors past biases (for example, undervaluing features that appeal to minority user groups if trained on data where those were deprioritized). These concerns align with the broader issue of fairness in AI: ensuring the technology does not inadvertently discriminate or propagate outdated norms. Mitigating bias in generative AI requires conscious effort, such as curating more diverse and representative training data, applying fairness-aware algorithms, and conducting regular audits of AI outputs for bias. For developers, this also means being alert to possible blind spots in AI suggestions – the team should question whether an AI-recommended solution might be biased or suboptimal and consider alternative perspectives rather than defaulting to the AI's way.

Another ethical dimension is accountability and transparency. When an AI assistant writes faulty code or produces an inappropriate output, who is responsible? In traditional software development, if a bug in a module causes a failure, the authors of that code (or their overseers) are accountable. With AI-generated code, the origin is a black-box model that cannot easily explain its reasoning. This lack of transparency makes it challenging to trace accountability. Developers might be tempted to blame the AI for errors, but ultimately it is the human team that decides to use those AI outputs. Thus, organizations must establish clear accountability frameworks: humans should always remain responsible for reviewing and accepting AI contributions. Ethically, one should treat AI as a tool. The final decisions and quality control should be with human engineers. This is especially important in safety-critical software (e.g. medical, automotive or financial systems) where an AI-generated error could have serious repercussions. Ensuring transparency is also part of the solution: keeping logs of AI suggestions, why certain decisions were made, and involving multiple reviewers can create an audit trail. Some industry guidelines even propose that AI systems in software development should be auditable and that their training data origins be documented, so that

any license or bias issues can be tracked.

2.4 Gaps in Current Research

While literature on AI-assisted software development is growing, several important gaps remain in the current research. One such gap is the limited empirical work focusing on startup-specific environments. Much of the existing knowledge about integrating AI into development comes from either large technology companies or general software engineering contexts. Startups, with their fast-paced and resource-constrained nature, have not been studied in depth – leaving questions about how generative AI adoption plays out in such settings largely unanswered (Marchesi et al., 2025). Only very recent efforts have begun to explore this space (e.g. identifying potential use cases of GenAI in startups (Marchesi et al., 2025)), but comprehensive, real-world studies are scarce. There is a lack of longitudinal research on the long-term impacts of AI assistance on development teams. Most studies and case reports so far examine short-term productivity boosts or immediate outcomes. We know little about what happens over months or years of continuous AI tool usage. For instance, do developers maintain their level of skill and critical thinking over prolonged AI reliance, or does a form of skill atrophy set in? How does team dynamics evolve as AI becomes a regular "team member" in the coding process? These are open questions, as longitudinal studies or long-term observations have not yet featured prominently in the literature. The need for such research is often noted – authors of an initial pilot study on AI integration concluded that their work "sets the stage for continued exploration" and that further research is necessary to address the limitations and open issues identified (Coutinho et al., 2024). There is no widely accepted framework for integrating AI across the software development lifecycle. Current research provides pieces of the puzzle (for example, studies on using AI in coding, separate studies on AI in testing, etc.), but a holistic framework or set of best practices that covers all phases of development is still lacking. The importance of seamlessly fitting AI tools into existing workflows is well recognized (Russo, 2024)– in fact, successful adoption of generative AI appears to hinge largely on compatibility with developers' current processes (Russo, 2024). Yet, guidance on how to achieve that compatibility (especially in smaller organizations) remains insufficient. There is a gap in translating the promising results seen in controlled experiments or isolated cases into an end-to-end methodology that practitioners can follow throughout project life cycles. This thesis seeks to help bridge some of the above gaps. Recognizing the lack of research on startups, the study reported in this thesis focuses explicitly on how startup software teams incorporate generative AI tools into their development practices. By observing and gathering data (e.g. through interviews) from real startup teams, it provides new empirical insights tailored to this context. These insights include practical guidelines and recommendations (as introduced in Section 2.2) drawn from direct experiences of startups, thereby extending the academic discourse into a domain that

operates differently from large enterprises. The research design emphasizes a more longitudinal perspective on AI adoption. Rather than just capturing initial reactions, it considers the evolving impact of AI assistance on team dynamics and individual developers' skills over time (for example, tracking whether prompt literacy and oversight practices improve as teams gain experience with AI, or whether reliance on the tool increases). While the timeframe of a master's thesis is limited, this work aims to at least lay groundwork for understanding long-term trends, which future studies can build upon (Marchesi et al., 2025) (Russo, 2024).

3 Research Methodology

This chapter describes the methodological approach of the study, detailing the qualitative research design, data collection through interviews.

3.1 Research Design and Rationale

This research adopted a qualitative, interpretivist approach to explore how generative AI is used in agile software startups. An interpretivist paradigm was chosen because the study seeks to understand subjective experiences and socially constructed meanings in a real-world context. The interpretivist stance is well suited for studying emerging practices (like AI adoption) where understanding people's behavior and decision making is crucial.

A qualitative approach was utilized. In this study, semi-structured interviews are utilized as the main method of gathering information. Semi-structured interviews enable a focused inquiry, where open-ended questions guide the conversation which allows participants to introduce new insights. This design choice is justified by the need to balance consistency across interviews with the freedom for interviewees to discuss what they find significant.

The rationale for using semi-structured interviews is also supported by methodological literature. Such interviews are characterized by open-ended questions and an interview guide covering broad topics. This format was chosen to elicit narrative responses and examples from participants, rather than simple yes/no answers. Open-ended questioning uncover how generative AI tools are integrated into workflows, how team members perceive their impact, and the rationale behind their usage.

3.2 Data Collection: Semi-Structured Interviews

An interview protocol was carefully developed to guide these conversations. The development of the interview protocol involved reviewing literature on agile development and generative AI to identify key themes and questions. An initial list of questions was drafted, focusing on areas such as: how teams integrate AI in their development process, changes in workflow or collaboration attributed to AI, challenges faced (e.g., code quality, trust in AI outputs), and perceived benefits or drawbacks. The protocol was then refined with input from the thesis supervisor and through a pilot test with a colleague (not part of the main study) to ensure questions were clear and open-ended.

The final interview guide consisted of open-ended questions grouped by topic. For example, one section of the guide focused on AI usage in daily tasks, with prompts like "Can you describe a recent instance where you used a generative AI tool (e.g., code assistant or

ChatGPT) during a sprint? What did you use it for, and how did it influence your work?" Another section explored team dynamics, asking "How has the introduction of generative AI tools affected collaboration or communication within your team?" By using open-ended formulations, the interviews encouraged participants to elaborate in narrative form, providing context, reasons, and stories.

The interviews were conducted one-on-one (virtually, via a video conferencing platform). Each interview lasted approximately 15–30 minutes, allowing sufficient time to cover all guide sections while giving participants room to introduce relevant insights. Interviews began with questions to put participants at ease (such as asking about their role and familiarity with AI tools) before moving into specific topics. The semi-structured nature meant that while all participants were asked about the same overarching topics, the order and depth varied.

All interviews were audio-recorded with the participants' consent and later transcribed to ensure accuracy. These transcriptions served as the primary data for analysis. Recording the interviews helped capture details in phrasing, and reviewing the transcripts allowed the researcher to become deeply familiar with the content before starting the thematic analysis.

3.3 Participant Selection

Participants were selected through the researcher's existing professional network, including individuals with whom the researcher had previously collaborated or maintained industry connections. A referral approach was also used, where some initial contacts recommended other suitable individuals, forming a mild snowball sampling strategy.

3.3.1 Recruitment

The inclusion criteria were: (1) currently or recently (within the past year) working in an agile software development setting (e.g., Scrum or Kanban) within a startup or scale-up context; (2) having experience using generative AI tools (such as code generation assistants, AI-powered debugging or planning tools) in their development work; and (3) holding a role that involved direct engagement with development workflows (such as developers, team leads, product managers, or engineering managers). Participants not actively involved in development activities were excluded. Proficiency in English was required, but there were no restrictions on geographic location. A total of eight participants (P1 through P8) were interviewed.

3.3.2 Participant Details

All participants remain anonymous, but an overview of their profiles is provided in Table 1. The table summarizes key attributes: role, company size, years of professional experience. No personal identifiers (such as names or company names) are reported. The roles represented include software engineers, product management, and technical leadership, reflecting a cross-section of agile team structures.

Company sizes ranged from very small startups (~30–50 employees) to somewhat larger scale-ups (~150–200 employees); all organizations can be classified as startups or early-stage growth companies—none were large enterprises. Participants' experience in the software industry ranged from approximately 2 to 10 years, providing a mix of mid-level and senior insights.

Table 1: Anonymized Participant Demographics

Participant	**Role**	**Company Size**	**Experience**
P1	Product Manager	45 employees (Startup)	2 years
P2	Software Engineer	∼100 employees (Scale-up)	4 years
P3	CEO	10 employees (Startup)	10 years
P4	Technical Consultant	15 employees (Startup)	3 years
P5	Software Engineer	80 employees (Startup)	4 years
P6	Product Expert	22 employees (Startup)	3 years
P7	Software Engineer	45 employees (Startup)	3 years
P8	Scrum Master	45 employees (Scale-up)	3 years

3.4 Data Analysis Method: Thematic Analysis

The interview data were analyzed using *thematic analysis*, following the six-phase framework articulated by Braun and Clarke (2006). Thematic analysis was selected because it provides a systematic yet flexible method for identifying patterns of meaning (themes) across qualitative data. It is a widely accepted approach that can yield a rich and detailed account of experiences. Braun and Clarke's method is theoretically agnostic, aligning well with the interpretivist stance of this study.

3.4.1 Braun and Clarke's Six Phases of Thematic Analysis

1. **Familiarization with the Data:** The researcher immersed in the raw data by transcribing interviews (where not already transcribed) and reading the transcripts multiple times. Initial observations and marginal notes were made during this stage to highlight potentially relevant insights. This phase involved repeated reading to actively search for meanings and patterns, laying the foundation for all further analysis.

2. **Generating Initial Codes:** With familiarity established, the researcher systematically reviewed each transcript and assigned descriptive codes to text segments relevant to the research questions. Coding was conducted manually with the assistance of NVivo 12 software for organization. Codes were often data-driven (in vivo) and aimed to remain close to participant language. For instance, the phrase "ChatGPT helps me finish tasks faster when I'm stuck" was coded as *"productivity aid"*. All potentially relevant data were coded inclusively in this stage.
3. **Searching for Themes:** The researcher grouped similar or related codes into candidate themes. For example, codes such as *"productivity aid"*, *"faster prototyping"*, and *"time-saving"* were clustered under the provisional theme *"Efficiency Gains from AI"*. This iterative phase involved using visual mapping and mind-mapping techniques to explore thematic relationships and refine groupings.
4. **Reviewing Themes:** Each candidate theme was evaluated to ensure internal coherence and external distinction. Themes were revised, merged, or discarded as necessary. For example, overlapping themes were combined, and underdeveloped ones were restructured. This phase helped finalize a coherent set of well-supported themes aligned with the dataset.
5. **Defining and Naming Themes:** Once finalized, each theme was clearly defined to articulate its scope and relevance to the research objectives. Sub-themes were also identified if appropriate. For example, *"Workflow Efficiency Gains"* was defined as "ways in which generative AI tools streamline or accelerate software development tasks in agile processes." Theme names were chosen to be concise, clear, and reflective of participants' views.
6. **Writing the Report:** The final phase involved producing a narrative that explains each theme with supporting evidence from the data. This process is reflected primarily in Chapter 4 (Findings), where each theme is elaborated with quotations and interpretation.

3.4.2 Thematic Analysis Process and Implementation

Figure 1 illustrates the six-phase thematic analysis process applied in this study, progressing from data familiarization through to final write-up. During the writing phase, for example, the researcher often revisited earlier stages to refine interpretations. This iterative approach helped ensure that the final themes were well-substantiated and representative of how generative AI is used and experienced by agile software teams.

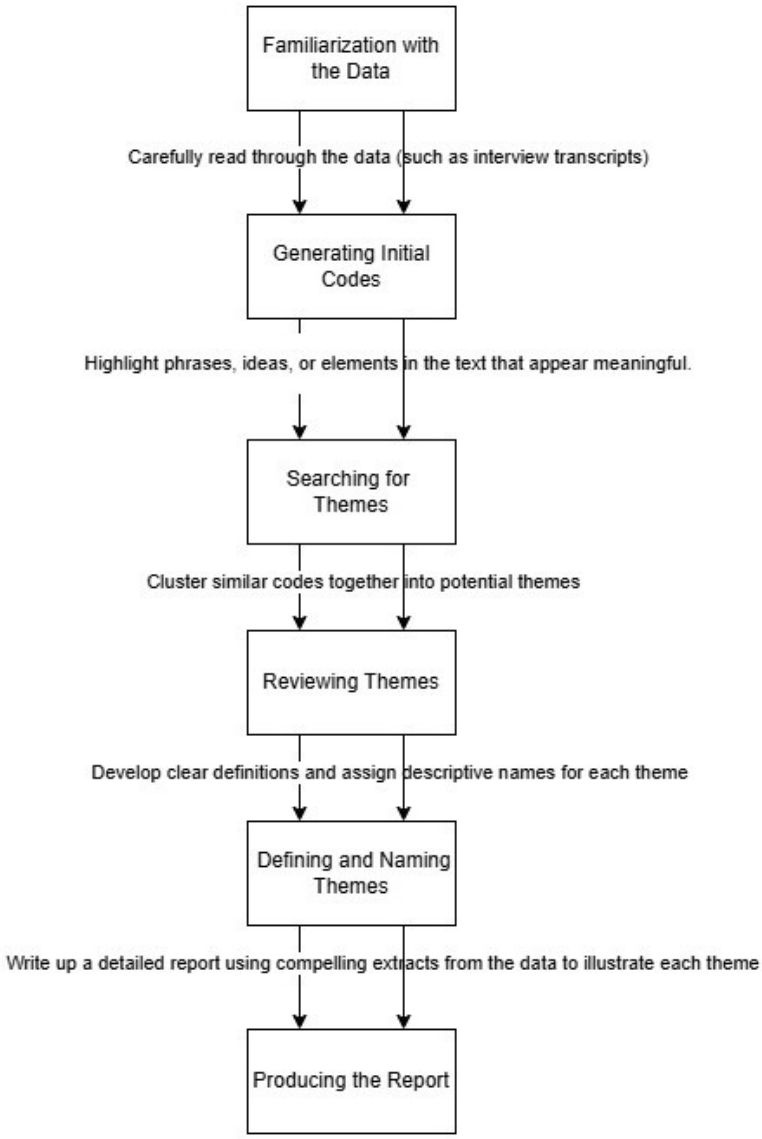

Figure 1: Schematic of Braun & Clarke's six-phase thematic analysis process, as implemented in this study.

Use of Thematic Analysis in this Study: By following Braun and Clarke's framework, the researcher ensured a structured and transparent approach to qualitative analysis. To illustrate how raw data were abstracted into higher-level themes, Table 2 presents a synthesized excerpt showing sample codes and the themes they contributed to.

Table 2: Example of Codes and Corresponding Themes (synthesized sample)

Theme	**Sample Initial Codes (examples from interviews)**
Workflow Efficiency Gains	"AI auto-completes boilerplate code" "Faster prototyping with AI suggestions" "Reduced debugging time thanks to AI"
Team Collaboration Dynamics	"Less pair programming needed when using AI" "AI recommendations spark team discussions" "Sharing AI findings in daily stand-ups"
Trust and Quality Concerns	"Need to double-check AI-generated code" "Wary of over-reliance on AI" "Ensuring code quality when AI involved"

Note: The table illustrates how specific codes derived from participant interviews were grouped under broader themes. For example, comments about time-saving effects of AI were abstracted under the theme *Workflow Efficiency Gains*, while remarks about changes in team communication informed the *Team Collaboration Dynamics* theme. Concerns related to reliability and cautious use of AI were grouped under *Trust and Quality Concerns*. This abstraction process is central to thematic analysis and demonstrates the link between raw data and final thematic structure.

By applying thematic analysis, the study was able to condense hundreds of pages of interview transcripts into a manageable set of main themes and sub-themes. This method maintained a strong link between the data and the findings: every theme is supported by multiple quotations. The rigorous following of Braun & Clarke's six steps (with careful documentation at each stage) also contributes to the trustworthiness of the analysis, as described in 3.6.

Thematic analysis enabled the researcher to reduce a large volume of qualitative data—comprising several hours of interviews and extensive transcripts—into a concise set of recurring themes and sub-themes. This structured method ensured traceability between original participant responses and analytic conclusions. The rigorous application of Braun and Clarke's six-step method, including detailed memoing and iterative coding, strengthens the credibility and transparency of the findings, as further supported in Section 3.6.

3.5 Ethical Considerations

The study was designed and conducted in accordance with LUT University's research ethics guidelines and general principles for research with human participants, ensuring respect for the rights and well-being of all involved.

3.5.1 Informed Consent

Prior to participating, each interviewee was provided with an information sheet outlining the purpose of the research, what participation entailed, and any potential risks or benefits. Participants were informed that their involvement was entirely voluntary and that they could decline to answer any question or withdraw from the study at any point without consequence. This consent process included explicit permission to audio-record the interview and to use anonymous quotations in the thesis. All participants were adults (over 18) and gave consent with full understanding of the research procedures.

3.5.2 Anonymity and Confidentiality

To protect participants' identities, all data were anonymized. As shown in Table 1, participants are referred to by codes (P1, P2, etc.) and only non-identifiable descriptors (like role or region) are used in reporting results. No real names, company names, or specific product details are disclosed in the thesis. During transcription, any mentions of identifiable information (such as a person, company or product name) were redacted or replaced with generic descriptors (e.g., "[a major cloud provider]" instead of naming the company).

3.5.3 Participant Review

After each interview was transcribed, a copy of the transcript (or a summary thereof) was offered back to the respective participant for review – a process often called member checking. Participants had the opportunity to correct any inaccuracies or clarify their statements.

3.5.4 Data Protection and Storage

In compliance with data protection regulations (such as GDPR, since some participants were in the EU), personal data was minimized and protected. Contact information of participants was stored separately from research data and will be destroyed after the study's completion. The anonymized research data will be retained for a certain period (as required for academic transparency or potential future research), but any such retention will also be under secure conditions.

In conducting this study, particular care was taken given the topic's context in professional life: participants were sharing experiences from their workplaces. They were advised not to reveal proprietary or sensitive company information during interviews, and the interviewer steered conversation away from any potentially sensitive corporate specifics.

3.6 Validity and Limitations

In qualitative research, especially under an interpretivist paradigm, the concepts of validity take the form of trustworthiness criteria. Lincoln and Guba's four criteria – credibility, transferability, dependability, and confirmability – were used to evaluate and bolster the trustworthiness of this study (Stahl & King, 2021) (QDAcity, n.d.). This section discusses how the study addressed these criteria, as well as the inherent limitations of the chosen methods.

3.6.1 Credibility

To enhance credibility, several strategies were employed. The researcher engaged in prolonged engagement and persistent observation with the data: spending time in interviews and in iterative reading of transcripts getting an in-depth understanding.

The use of member checking (as detailed in Section 3.5). allowed participants to verify that their statements were captured correctly, helping align the findings with participants' realities.

A peer (fellow student) was involved in peer debriefing – he reviewed a subset of transcripts and the preliminary coding to provide an external check on interpretations. This process helped challenge any researcher biases and refine theme definitions.

3.6.2 Transferability

This thesis has attempted to provide detailed descriptions of the research context, participants, and analytic process, as well as illustrative quotes in the findings, so that the boundaries of the findings are clear. For instance, the context (agile teams in software startups using generative AI) is described in detail; readers interested in, say, large enterprise settings or non-agile teams can judge how the results may differ in those environments. While the sample is not statistically representative, the insights may matches with similar settings – e.g., other tech startups undergoing digital tool adoption.

3.6.3 Dependability

To ensure the study's process was logical, traceable, and documented, an audit trail was maintained. The audit trail includes documentation of the research design decisions, interview guides, raw data (transcripts), coding book, theme development notes, and revisions. By keeping a reflexive journal, the researcher noted methodological choices and changes (for example, if any interview question was added after early interviews, it was recorded). The step-by-step following of Braun & Clarke's framework also adds to dependability, as it provides a structured procedure that others can follow or evaluate (Nowell et al., 2017).

3.6.4 Confirmability

In this interpretivist study, complete objectivity is neither attainable nor expected, since the researcher is the instrument of analysis. However, steps were taken to ensure findings are grounded in data. All themes are supported by direct quotes from multiple participants (to be presented in the Findings chapter), demonstrating the link between raw data and interpretations. The researcher practiced reflexivity, acknowledging and bracketing personal biases as much as possible. Regular discussions with peers and the supervisor about the coding decisions also served to keep the analysis anchored in what the participants actually said, rather than what the researcher expected to find.

3.6.5 Limitations

Sample Size and Composition

With 8 participants, the study's sample is relatively small. While appropriate for qualitative depth, this means the findings should be regarded as exploratory. Important perspectives might be missing – for instance, if none of the participants experienced a certain challenge, that challenge won't appear in our themes but could exist in other teams.

Contextual and Temporal Limitations

The research was conducted at a time (2024–2025) when generative AI tools (like GitHub Copilot, ChatGPT) were relatively new in mainstream use. The findings are a snapshot of experiences during this early adoption phase. This means there may be novelty effects at play (teams experimenting in ad-hoc ways) and the landscape is rapidly evolving. If the same study were done a year or two later, as AI tools mature and guidelines/best practices emerge, results might differ (some current challenges might be resolved, new issues might appear).

Single Data Source (Interviews)

The study relies solely on self-reported data from interviews. This raises the possibility of common method bias and issues like participants' retrospection inaccuracies or social desirability in responses. Participants might have overstated either the positive effects of AI (to appear tech-savvy) or negative effects (if they are skeptical of AI), or they might simply not recall certain details about their AI usage. The absence of triangulation with other data (e.g., observations of team meetings, analysis of code repositories to see AI contributions, or surveys) means the findings rest on what participants said. While what people say is valuable and indeed the focus here (perceptions), it might not always align perfectly with what they do.

4 Findings and Thematic Analysis

4.1 Overview of Participants

Eight professionals from generative AI-assisted software development teams participated in this study, anonymized as P1 through P8. Their details, including roles, company size, and years of experience, have already been presented in Table 1 (Section 3.3.2).

The participants represented a variety of roles including software engineers, product managers, a startup CEO, and a technical consultant—covering both technical and strategic perspectives. All worked in startups or small companies actively experimenting with generative AI tools. Company sizes ranged from under 15 employees (early-stage) to over 100 (scale-ups). Most participants had 3–5 years of experience, with one (P5) having 10 years of experience.

P1 and P2 provided insights on bridging business and technical work from a product management perspective. P3, P4, P7, and P8 offered deep developer perspectives on AI-assisted coding, testing, and collaboration. P5, as a CEO, discussed adoption from an organizational strategy viewpoint, while P6 contributed a consulting lens.

4.2 Summary of Coding and Theme Development

The interview data were analyzed using a thematic analysis approach (as described in Chapter 3). In the first coding cycle, numerous codes were identified for recurring concepts – for instance, participants frequently mentioned ideas like automating code reviews, improving documentation, saving time, or challenges with AI suggestions. These granular codes were then reviewed and clustered during axial coding to form coherent categories. Through iterative refinement, two overarching thematic categories emerged: *benefits of generative AI in the team context*, and *challenges associated with its use*. Within each category, several sub-themes were defined to capture more specific facets of participants' experiences.

Table 3 and Table 4 illustrate how the initial codes informed the final theme structure. Table 3 focuses on the benefits, listing each sub-theme and examples of coded concepts that constitute it. Similarly, Table 4 outlines the challenge sub-themes with representative codes. This mapping demonstrates the progression from raw interview data (codes) to abstracted findings (themes).

Many codes were mentioned by multiple participants, indicating a strong consensus on key issues. For example, codes related to *"time-saving"* and *"productivity"* were ubiquitous

and ultimately informed several benefit themes, whereas mentions of *"incorrect AI outputs"* and *"data privacy"* were common threads underlying multiple challenge themes. The final themes and sub-themes serve as the organizing structure for the findings in (Section 4.3, 4.4).

Table 3: Development of Benefit Themes from Codes

Benefit Sub-Theme	**Example Initial Codes**
Refactoring with AI Tools	Code suggestions; Code refactoring; Automated code formatting; AI-assisted code review
Enhancing User Story Clarity	Generating user stories; Requirements drafting; Clarifying specs with AI
Automating Daily Standups	Meeting transcription; Stand-up summary generation; AI note-taking for meetings
AI-Supported Testing and Debugging	Auto-generated test cases; Bug detection; Performance issue spotting; QA automation
Cross-Team Documentation Sharing	Documentation automation; Knowledge base creation; Summarizing technical docs for non-devs
Continuous Learning and Upskilling	On-demand tutorials; Learning new frameworks via AI; Skill improvement with AI help
Strategic Time Reallocation	Time saved on routine tasks; Focus on creative work; Increased velocity; Productivity gains
Empowerment of Non-Technical Staff	Non-developers using AI; AI-assisted content creation (e.g., teachers, marketing); Collaboration across roles via AI

Table 4: Development of Challenge Themes from Codes

Challenge Sub-Theme	**Example Initial Codes**
Tool-Project Mismatch	Integration difficulties; Tool incompatibility; Unsupported tech stack; Setup/configuration issues
Hallucinated Functions & Suggestions	Incorrect code suggestions; False or non-existent references; Unreliable outputs; AI "hallucinations"
Data Privacy Concerns	Confidential code exposure; Privacy risks; Data leakage; Intellectual property issues
Over-Reliance and Dependency	Overdependence on AI; Blind trust in AI output; Neglecting code review; Need for human oversight
Reduced Peer Interaction	Less developer-to-developer discussion; Fewer team brainstorming sessions; Communication becoming impersonal
Prompt Skill Requirement	Need for prompt engineering; Learning curve with AI tools; Crafting effective queries; Trial-and-error in prompts
Cultural Resistance	Resistance to change; Skepticism about AI; Fear of job displacement; Preferring manual methods

By examining the relationships between these codes and themes, the analysis ensured that each theme is grounded in multiple pieces of evidence from the interviews. The following (Section 4.3, 4.4) present the findings in detail, structured by these sub-themes. Direct quotes from participants (attributed as P1–P8).

4.3 Benefits of Generative AI in Startups

Participants reported improvements in code quality and development speed, better documentation and knowledge sharing, and even positive shifts in how team members allocate their time and learn new skills. The following sub-sections represent details of these benefits. Each sub-theme is supported by interview evidence, with participant quotes.

4.3.1 Refactoring with AI Tools

One clear advantage noted by developers was the ability to use AI for refactoring code and implementing quick improvements. Several participants described how AI-powered coding assistants (such as GitHub Copilot or AI-integrated IDEs) help streamline the process of improving existing code.

For example, Participant 3 (P3) explained that his team initially applied AI to small-scale code edits and clean-up tasks:

> "We started using AI for things like note-taking and documenting sprint decisions, things that wouldn't break production."

By beginning with minor refactoring and code formatting tasks, they built trust in the AI's capabilities before moving on to more complex use.

Developers soon found that these tools could handle larger code transformations as confidence grew. As Participant 4 (P4) reported:

> "Initially, I just used it for suggestions and minor fixes. Now I can use it for whole features — but that came after testing it in smaller ways."

This progression from trivial fixes to substantial coding work was common across the interviews. Participants described scenarios where they would highlight a block of code and ask the AI to optimize or rewrite it. P3 noted the convenience of an AI-augmented IDE in this context:

> "You can highlight a line [of code] and say, 'write this for me,'... Inside that application, you can directly ask the AI."

Using such tools, developers were able to restructure legacy code, enforce consistent coding standards, and eliminate code smells more efficiently. AI-assisted also improved code quality and consistency. Participants mentioned that the AI often suggested standard design patterns or best practices during refactoring.

Participant 8 (P8) shared an illustrative example involving the use of an AI code review tool called *DeepCode*, which flagged a critical performance issue in their system. This led to a timely and effective resolution. As P8 described:

> "[It] saved us... we had some performance issues... DeepCode found that and luckily we were able to resolve that."

This example shows the value of AI in detecting non-trivial issues and promoting higher standards in code quality during the development lifecycle.

4.3.2 Enhancing User Story Clarity

Another theme that emerged was the use of AI to improve the clarity and completeness of user stories and requirements. Participants in product management roles (P1, P2) found that generative AI could assist in drafting and refining requirements documentation.

P2, for instance, described a shift in his workflow for writing user stories:

> "Before, I was manually creating documentation, requirements and user stories… now I ask [the AI] to create user stories or a requirements document for me and then I just do a few changes."

This practice has allowed product owners to produce well-structured requirement specifications with less effort.

P6 observed that when an AI tool generates a first draft of a user story or specification, it often enforces a logical structure and clear language. This makes it easier for both developers and non-technical stakeholders to understand the feature intentions. As P6 noted:

> "It's easier for the product owner or the PM to read AI-generated documentation — it's clearer."

Participants also mentioned that AI could incorporate acceptance criteria and edge-case scenarios into user stories that the author might otherwise miss. P1, for example, sometimes used ChatGPT as a sounding board to brainstorm possible user flows or corner cases for a given feature, thereby enhancing the story's completeness.

4.3.3 Automating Daily Standups

Participants reported that AI tools have started to automate parts of these routine communications. Several developers (P3, P7, P8) mentioned using AI-based meeting assistants to record stand-up meetings, transcribe discussions, and even summarize the updates from each team member.

P8's team, for example, employs AI transcription services in every meeting:

> "We… take the transcripts and also make meeting notes using those tools."

By having an AI automatically capture what was said, the need for manual note-taking during standups is eliminated.

The automation of stand-up documentation yields two main benefits. First, it ensures that accurate records of daily updates and blockers are preserved. These can be easily shared across the team or with absent members, improving transparency. Second, it has made the stand-up meetings themselves more efficient. P1 noted that since adopting an AI note-taker, their stand-ups have become shorter and more to-the-point, because the tool can summarize repetitive updates.

In fact, some teams have experimented with asynchronous stand-ups: team members input their daily status into an AI-assisted chat, which then produces a consolidated stand-up report for everyone.

Participants also reflected on how this affects team communication. According to P2, AI-curated stand-up summaries save time but require the team to deliberately maintain personal interaction in other ways so that team cohesion is not lost. As P2 remarked:

> "Meetings are now shorter... I use my time for decision making... rather than taking notes."

However, he cautioned that one must balance efficiency with human connection—a point discussed further in Section 4.4.5.

Automating standups and meeting notes frees up developers and managers from clerical tasks and ensures that important decisions or impediments raised each day are documented and disseminated quickly. As P1 summarized:

> "These AI tools make your life easy by handling the mundane aspects of daily coordination."

4.3.4 AI-Supported Testing and Debugging

Perhaps one of the most frequently cited advantages of generative AI was its contribution to software testing and debugging. Participants across technical roles described how AI tools accelerate testing processes, help catch bugs, and improve code quality assurance.

P4 explained that his team began by using AI in low-risk areas like test case generation and code review, which:

> "Allowed the team to build trust in the tool."

Over time, this expanded into extensive use of AI for writing unit tests, generating test data, and even performing static analysis. For example, P8 recounted using an AI service to au-

tomatically generate dozens of unit test cases for a given module, significantly reducing the manual effort required to cover different scenarios.

A major benefit of AI in this context is early bug detection. P8 noted that an AI-powered testing tool:

> "Catches errors early... which means that I can focus more on solving the complex problems"

P4 mentioned that using GitHub Copilot for code suggestions often prevent bugs by suggesting more refined code in the first place. Participants cited tools like *DeepCode* and *Amazon CodeGuru*, which use AI to perform code reviews and flag potential issues (ranging from security vulnerabilities to performance concerns) automatically during development. As P8 described:

> "Consistent AI feedback on code quality made it easier for us to discuss potential improvements and align on best practices."

AI assistance in debugging was another highlight. When facing an error or an exception, developers like P3 and P4 would sometimes query ChatGPT with the stack trace or a description of the issue. The AI could often point them toward possible causes or known solutions in a fraction of the time it would take to manually search documentation. P3 gave an example of an obscure error that the AI identified as a known library conflict, saving the team hours of troubleshooting.

Participants emphasized that the AI's debugging suggestions had to be verified, and they would cross-check any AI-proposed solution. However, as a starting point, it was invaluable. Overall, AI-supported testing and debugging have improved code reliability and team productivity. P4 summarized the impact succinctly:

> "It saves us time by automating the repetitive tasks like testing and code generation and also by catching the bugs early... [which] help us build the better software really faster."

4.3.5 Cross-Team Documentation Sharing

Generative AI has also enhanced documentation practices, making it easier to share knowledge across different teams and roles. Several participants noted that AI tools can generate and format documentation that serves both technical and non-technical stakeholders, thereby bridging communication gaps.

P1's organization, for example, uses AI to document project processes and user feedback:

> "We use AI to… document all these processes and document user feedbacks."

By doing so, updates from customer support or user research (which are in natural language) can be refined into concise reports for the development team.

Participants found that this AI-assisted documentation is faster and often more consistent than if written manually under time pressure. P5, who leads an EdTech startup, shared that they leveraged AI to create technical documentation and API guides for internal use, which they could then share with a new hire or a cross-functional partner (like a curriculum designer). The AI would ensure the documentation had a uniform style and covered key points (e.g., function usage examples, input/output formats), requiring only minor edits from an engineer for accuracy. This practice has shortened the turnaround time for producing documentation whenever a new feature is completed.

The sharing aspect is crucial in startups, where one person often wears many hats and explicit knowledge transfer is needed. P6 pointed out that clear AI-generated docs meant that:

> "People from marketing or customer support [could] use AI to write drafts or understand technical stuff."

In one case, P6 described how a marketing team member used an AI tool to generate a draft of a technical blog post based on the engineering documentation of a feature. The engineer (P6) only had to review it for technical correctness, rather than writing it from scratch.

This example illustrates how AI can enable non-engineering staff to engage with technical content more effectively, ultimately improving cross-team collaboration. It reduces the reliance on engineers to personally explain or translate technical details every time.

4.3.6 AI as a Tool for Learning and Skill Development

Many participants credited generative AI with helping them learn new technologies and improve their skills on the job. In the fast-paced startup setting, developers often need to pick up unfamiliar frameworks or solve novel problems quickly. In this regard, AI acted as a real-time mentor or tutor.

> "I have personally taught myself a lot using AI," said P6, explaining that he can find tutorials and examples for almost any topic by querying ChatGPT.

Rather than searching through numerous web pages or documentation files, developers would

ask the AI direct questions and get explanations or code snippets in return. P6 emphasized:

> "I can get help with something that I do not know how to do… and I can always find examples [with AI]."

Participants also pointed out that AI provides multiple solution approaches, which enhances their understanding. P3 mentioned that when experimenting with a new algorithm, he asked the AI for different possible implementations:

> "It will provide you 10 solutions… a lot of [those] are obviously garbage as well. But since you have the knowledge, you know what is best… So in that case, you can use it and it's helping you, making you more innovative."

AI can generate a variety of ideas, and the developer learns by evaluating which of those ideas are relevant. .

Beyond coding, participants in product roles also used AI to upskill in adjacent domains. For instance, P1 (a product manager) used AI to learn about agile estimation techniques and innovation frameworks by having dialogues with the AI as if it were an expert. Since the AI can tailor its answers to the context provided, it's like having a customizable textbook or coach. P1 said this helped him introduce better practices in his team drawn from industry knowledge that he accessed via ChatGPT.

All participants agreed that despite AI's support, self-directed practice and verification remained important. The AI could occasionally produce incorrect information, so they treated it as a guide rather than an oracle. Still, the efficiency of AI in teaching was undeniable. P5 remarked that even for non-technical staff at his startup.

> "AI makes it easier… to contribute without requiring deep technical expertise,"

since they could ask the AI to explain technical concepts in simple terms.

4.3.7 Strategic Time Reallocation

A recurring benefit highlighted by participants was that AI tools free up developers' and managers' time, allowing them to reallocate effort to more strategic and creative tasks. By handling tasks such as coding boilerplate, writing routine documentation, or running tests, AI enables teams to focus on higher-level responsibilities.

P2 illustrated this point by describing how his day-to-day work has shifted:

> "By automating my day-to-day tasks. . . , I get a lot of free time to be innovative and think of new ideas. . . because of the time that it has saved me by creating the summary of a meeting."

Instead of spending an hour compiling meeting notes, P2 now uses that time to brainstorm product improvements. This efficiency gain has the potential to spur innovation.

Several developers similarly noted that they could invest more time in design and architecture, which are considered higher-value activities, once AI handled the grunt work. P4 explained that with AI-generated tests and code reviews in place, he was able to focus on optimizing the system's architecture and performing performance tuning that had previously been postponed. P8, an engineer, echoed this sentiment, saying:

> "It saves us time. . . and also by catching the bugs early,"

Non-technical tasks also saw reallocation. P1 mentioned that their team reduced the frequency and length of certain meetings, as AI could summarize updates, and they redirected that time into creative discussions and strategic alignment sessions.

P5, the CEO, observed a similar dynamic at the organizational level: with AI handling some content creation and support queries, employees had more time to dedicate to core product development and user engagement efforts.

Participants perceived this time reallocation as improving both productivity and job satisfaction. P2 expressed that his role changed "in a positive direction" because:

> "I get more time to. . . think of new ideas instead of. . . writing meeting notes. . . I now get to do the real work."

The outcome is that startups can move faster and operate more intelligently, delivering features sooner without overworking the team, because AI is augmenting their capacity. These findings align with the agile principle of maximizing the amount of work not done. By delegating low-level tasks to AI, teams can produce more strategic outputs with the same level of effort.

4.3.8 Empowerment of Non-Technical Staff

One of the most interesting benefits noted was the empowerment of team members in non-technical roles (or with less technical expertise) to contribute more directly in the software development process using AI tools. Traditionally, activities like writing specifications, ana-

lyzing data, or prototyping have required significant technical know-how. However, participants observed that AI is lowering these barriers.

P5 shared how in his startup, even educators with limited programming skills started using the company's generative AI features to produce content:

> "We are using AI in material creation... to help teachers create interactive tasks and exercises."

By incorporating AI, tasks that would have required a developer—such as generating interactive language-learning exercises—can now be done by the teachers themselves through a user-friendly AI interface.

Similarly, within development teams, roles like quality analysts, designers, or support specialists have begun using AI. P7 remarked that at his company:

> "Even people from marketing or customer support use AI to write drafts or understand technical stuff."

What this means is that a customer support agent, for instance, can use ChatGPT to help draft a troubleshooting guide based on technical documentation, or a marketing person can use it to interpret release notes and create user-friendly announcement copy.

Participants P6 and P3 also discussed how product owners and business analysts use AI to improve their work. P6 pointed out that when a product manager isn't from a coding background, AI can assist them in generating structured tech specs or SQL queries for data analysis. In fact, P5 recounted an example:

> "I can now write structured dev specs thanks to AI,"

said one product specialist in his team.

Overall, the data suggests that generative AI is fostering a more inclusive development culture. By serving as a technical assistant available to everyone.

Of course, companies like P5's have set some guidelines (e.g., on what not to do with AI, to prevent misuse), but the overall direction is enabling. As P6 put it:

> "AI helps bridge the gap between technical and non-technical roles on the team."

4.4 Challenges of AI in Startups

While participants were enthusiastic about the benefits of generative AI, they also encountered significant challenges and concerns when integrating these tools into their workflows. The adoption of AI is not without friction, with issues ranging from technical limitations of the tools to human and organizational factors. In the following subsections, we examine the key challenges identified in the interviews. Each challenge theme is supported by direct evidence from participants, highlighting the nuanced ways in which AI can complicate collaboration, trust, and productivity in a startup setting.

Notably, many challenges are interrelated with the benefits discussed earlier; for instance, the time-saving aspect of AI comes with a trade-off in how team communication occurs. Understanding these challenges is crucial for developing best practices (addressed in Chapter 5)

4.4.1 Tool–Project Mismatch

One practical challenge teams faced was a mismatch between AI tools and the specific project or tech stack in use. In some cases, the AI tools did not integrate well with the team's development environment or were not suited to their programming languages or frameworks.

P2 recounted difficulties when trying to incorporate an AI service into their product:

> "We were integrating ChatGPT [into our app], and we had like a lot of issues... there was no good documentation and [no details] regarding how they will be using our data."

This integration complexity slowed down development and introduced uncertainty, as the team struggled with poorly documented APIs and unclear behaviors of the AI component.

Participants also mentioned workflow disruptions due to tool limitations. P7, who tried an AI-based project management assistant, found that it didn't mesh with their existing agile tools: the AI would create tasks or comments that didn't sync correctly with Jira, causing confusion. As P1 explained:

> "Sometimes integration of AI tools becomes difficult because you're not able to vet the results."

In this case, the AI's outputs didn't fit neatly into their process, and it was hard to tell whether it was doing more harm than good.

4.4.2 Hallucinated Functions and Suggestions

A frequently reported issue was the AI's tendency to hallucinate—producing outputs that are plausible-sounding but incorrect or nonexistent. Developers encountered cases where the AI would generate code invoking functions or libraries that "don't exist," essentially fabricating solutions that looked legitimate at first glance. As P1 put it, the AI would sometimes confidently suggest invalid approaches.

P8 described this as getting:

> "Garbage in a response... when you ask something and you get something... I would say like garbage in response. That was actually a major issue with the LLMs."

Such hallucinations can mislead developers, especially if they are less experienced or trying something new and are not sure what the correct answer should look like.

These incorrect suggestions manifested in various ways. Sometimes, the AI would confidently provide a code snippet using an API call that did not exist in the relevant library version. P4 recounted a situation where Copilot suggested a function, optimizeDatabase(), which sounded useful but was entirely made up. It took the developer some time to realize that the function was hallucinated, as it closely resembled something that could plausibly exist.

A common challenge was that AI tools sometimes gave answers that looked correct but included functions or libraries that didn't actually exist. This caused confusion and wasted time. P1 experienced this:

> "One of the tools gave me a function for validation that didn't even exist in the library."

The consequences of these hallucinations include wasted time debugging non-issues and the potential introduction of bugs. A few participants admitted that early on they sometimes trusted the AI output too much. P3 mentioned integrating an AI-suggested code block without full verification, only to have the build fail. Reflecting on this, P8 said:

> "It gave me a method name that sounded real... I spent like 30 minutes trying to import it before realising it was fake."

This highlights that not only do hallucinations produce non-existent content, but even when

the code exists, it might be suboptimal or outdated because the AI's training data could contain flawed examples.

To mitigate this, participants developed a more skeptical approach to AI suggestions, often treating them as drafts that needed to be validated. P4 and P8 both stressed the importance of running tests on AI-generated code and avoiding the assumption that it was correct. They also learned to craft better prompts by providing more context, which reduced the chances of the AI filling in gaps with inaccurate or nonsensical information.

This ties into the prompt engineering skill discussed later (Section 4.4.6). Nonetheless, hallucination remains a core challenge inherent to current generative models. Even with careful prompting, as P6 noted:

> "If you don't give [a] detailed prompt, it can easily go wrong."

The AI might guess, and that guess can turn out to be a hallucination.

4.4.3 Data Privacy Concerns

A serious challenge that arose is the concern over data privacy and security when using AI tools. Many generative AI services (e.g., cloud-based code assistants or chatbots) require sending code or data to external servers for processing. Participants voiced apprehension about the implications of this, especially when proprietary or sensitive information is involved.

P8 was particularly direct about his policy:

> "If the feature touches something sensitive—like payment or data storage—I don't let AI near it."

This highlights a precaution to exclude critical or sensitive code from AI prompts, out of fear that the data could leak or be stored in the AI's training logs.

Some participants were worried about the risk of sharing sensitive information with AI tools like ChatGPT. One person even shared a real example of a data breach at their company, caused by someone using AI carelessly.:

> "I think I remember an example from my company. So last year we had a data breach because one of the employees was using GPT to get his work done quickly, but he was also typing his critical information... and some other employee also entered his credentials to get access to the GPT... they sort of leaked

> the critical information that was only to be used by the employees."

Concerns were not only about code but also about business data. P5, working in an education-related startup, mentioned his team's internal rule:

> "We advise teachers not to paste any textbook content or full lesson plans into AI—they could be copied later by someone else."

The interviews revealed that trust in AI providers is low when it comes to data handling. Some companies—especially those handling user personal data or operating under strict compliance regimes—outright ban using external AI services with any real data.

As a result of these privacy concerns, teams implemented certain best practices. They limited AI usage to non-sensitive areas (as noted in the "low-risk tasks" approach). They also explored self-hosted AI solutions for critical code, albeit those are still maturing. P8's stance of completely keeping AI away from payment or authentication code was echoed by others who identified specific domains that remain strictly human-only.

4.4.4 Over-Reliance and Dependency

Even as AI tools become more capable, participants warned of the danger of over-reliance on them, which can erode developers' own skills and judgment. The convenience of AI suggestions sometimes led team members to trust the AI too much or use it as a crutch. P6 observed this in junior developers especially: if Copilot or ChatGPT provided a snippet that "worked," they might include it without fully understanding it. As P2 cautioned:

> "Even after getting the right response, when you run it or execute the code it's actually wrong… so you cannot completely rely. You always need to recheck your code afterwards."

P3 described an effort he made to sometimes "turn off" the AI assistant for a day to ensure he could still code proficiently without it. This personal check-in was prompted by the realization that he was accepting AI suggestions too readily. As he put it:

> "You cannot completely rely [on it]."

The AI might suggest a solution that works now but isn't optimal long-term, or it might miss context that a human would consider.

Another aspect of over-reliance is the loss of critical thinking in design decisions. P2 mentioned a scenario where an AI proposed a design pattern for a feature. It seemed reasonable

and the team implemented it, but later they discovered it wasn't the best fit and had to refactor significantly. In retrospect, they realized they hadn't engaged in the same level of design discussion they normally would; the AI's quick answer short-circuited their deliberation. P3 reflected:

> "I think if I continue like this, I'll lose my habit of thinking through problems properly."

P6 said:

> "If someone just copies whatever ChatGPT gives, they're not learning anything. It makes you lazy"

If a developer always uses AI to generate boilerplate code, they might struggle to do it manually when needed or to debug it deeply. P1, from a product perspective, also worried that if team members rely on AI for tasks like writing meeting notes or test plans, they might not develop those communication or planning skills as much as they normally would. Essentially, over-reliance could lead to gaps in human capability that are only noticed when the AI is unavailable or fails.

4.4.5 Reduced Peer Interaction

A tool meant to enhance productivity can sometimes diminish team communication and collaboration, as reported by multiple participants. With AI assistants readily available to answer questions or provide guidance, developers felt less need to consult their peers as frequently as before. P4 observed a shift in his team's communication patterns:

> "Yeah, I believe that AI has led to reduced communication since we are, if we have an issue, we ask the AI first and if it gives a good enough answer, we just go with that. Whereas previously we might have asked a colleague first about some issue.."

This was said in the context of AI tools handling routine questions and generating updates, which on the surface is efficient—but it also meant fewer opportunities for spontaneous discussions and knowledge exchange among team members.

Several participants reflected on this reduced peer interaction with mixed feelings. On one hand, fewer interruptions can mean more focus time (an immediate productivity win). On the other hand, those informal interactions often serve to build team cohesion and spark collective problem-solving. P2 noted that before AI, a junior developer might ask a senior

for help with a bug, and through that conversation, not only would the bug get fixed, but the junior might learn broader debugging strategies and the team might identify an area for refactoring. Now, if the junior simply asks the AI and gets an answer, the bug is fixed quietly—but an opportunity for mentorship and team insight is lost.

4.4.6 Prompt Skill Requirement

Effectively using generative AI turned out to be a skill in itself—one that not all team members initially had. Participants identified the need for prompt engineering skills as a challenge, since getting useful outputs from an AI often depends on how the query or request is phrased. P5 pointed out that:

> "So you have to prompt very specifically. If you don't guide it well, it just makes up things or gives something too generic."

This became apparent when some team members would get nonsensical answers from the AI simply because the question was vague or missing context. There was a learning curve to formulating prompts that are clear, specific, and provide the right background information. P7's said:

> "Sometimes I get wrong answers because I didn't ask properly. It's not the AI's fault, but how I phrased it."

The challenge is not uniform across all team members. Typically, those with stronger communication skills or English proficiency adjusted more quickly, whereas others struggled initially. P3 noted that one of their senior engineers, while extremely competent in coding, often wrote very terse prompts and received poor results, leading him to dismiss the AI as "not useful." In reality, the issue was the prompt. P3 coached him to include more detail, after which the senior engineer started seeing the value. As P6 explained from his own trial and error:

> "If your prompt is not clear, the AI just confuses you more. You need to know what to ask, and how."

Despite improvements, participants acknowledged that prompt crafting remains an ongoing learning process. As AI models update and gain new features, the strategies for interacting with them continue to evolve.

4.4.7 Cultural Resistance

A challenge that surfaced in some interviews was resistance to AI adoption from certain team members or stakeholders, rooted in culture and mindset. Not everyone was immediately on board with using generative AI in software development. P1 observed:

> "Some people do not want to use AI tools… I have seen people who would not like to use AI tools, rather they would do everything manually."

This resistance sometimes came from senior engineers who trusted their own methods more, or from team members worried that AI might replace jobs or diminish their role.

Participants described several dimensions to this cultural pushback. One was fear and uncertainty, as the notion that "AI is taking people's jobs" had entered the team's discourse. In a startup environment, where job security is already tenuous, some developers were concerned that embracing AI might accelerate their own redundancy. P5 addressed this concern during a company meeting, emphasizing that AI was intended to assist rather than replace, and that the company viewed it as a tool to amplify human creativity. He even framed it as a way to "stay in the game" competitively—an innovation they needed to adopt thoughtfully, not fear.

Another aspect was simply personal work style preference. P6 shared that one of his colleagues, an experienced consultant, dismissed the AI outright because:

> "My senior… he is a bit older, in his 50s, and he said that ChatGPT [is] just a waste of time… [because] the user interface was not good enough for him."

Startups also had to consider client or management attitudes. P4, working in a service-based company, mentioned that a few clients were not comfortable with the idea of AI-generated code in their projects, fearing liability or quality issues. Thus, even if his team wanted to use AI, they had to sometimes limit its use or at least be discreet, to maintain client confidence.

4.5 Summary of Tool Usage

Participants in this study employed a range of generative AI tools, with ChatGPT and GitHub Copilot being the most prominent, alongside some use of specialized code analysis tools like DeepCode (now Snyk Code). Table 5 summarizes the tools, their primary functions, and how participants (P1–P8) utilized them in practice.

The overarching pattern is that participants initially applied AI to relatively contained tasks, such as generating small code snippets, automating documentation, or suggesting test cases.

As their confidence grew, they gradually scaled up to more complex uses. Common use cases included code generation (drafting functions or boilerplate code), code refinement (bug fixes or refactoring suggestions), and documentation support (summarizing or producing technical documents).

Participants across different roles found value in these tools. Developers used Copilot within the IDE for rapid coding, while non-developers leveraged ChatGPT in the browser for planning and documentation. In nearly all cases, participants reported some productivity improvements, especially in terms of time savings on routine tasks and ease of accessing information.

Table 5: AI Tools Used, Functions, and Participant Use Cases

AI Tool	**Primary Function**	**Participant Use Cases**
ChatGPT (OpenAI)	Chat-based language model for code and text generation; provides explanations, suggestions, and documentation support.	P1 – Drafting sprint notes; P3 – Debugging and code explanations; P5 – Educational content and design specs; P6 – Clarifying technical concepts; P7 – Problem-solving via Q&A.
GitHub Copilot	AI code assistant in IDEs (e.g., VS Code); suggests code completions and entire functions contextually.	P2 – Unit test generation and boilerplate; P4 – Feature implementation and refactoring; P7 – Routine code snippets; P8 – Task automation, excluding sensitive modules.
DeepCode (Snyk)	AI-powered static code analysis; detects bugs and suggests improvements based on learned best practices.	P4/P7 – Reviewing AI-generated code for bugs and standards adherence; enhancing code quality and reducing manual reviews.

5 Discussion

5.1 Reflection on Key Findings

Chapter 4 outlined both the opportunities and challenges that emerged from incorporating generative AI into software development team workflows. In this section, we reflect on those key findings, highlighting four major themes: improved productivity, documentation support, trust in AI, and shifts in collaboration dynamics.

5.1.1 Improved Productivity and Efficiency

The clearest outcome from the findings is that generative AI tools can improve developer productivity for well-scoped tasks. Participants widely reported time savings and efficiency gains when using AI for coding and related activities. Routine programming tasks—writing boilerplate code, trivial functions and repetitive sections of code—were completed faster with AI assistance.

P4's experience exemplified this; initially he only used the AI for "minor fixes" but in time trusted it to help implement "whole features," indicating that as the team grew comfortable, AI moved from the periphery of development to a more central supporting role. This evolution has accelerated development speed: With Copilot's real-time suggestions, P4 said that now he could produce code with fewer keystrokes and less context switching, and P2 noted that generating AI test cases freed up time to focus on designing more complex test scenarios manually.

AI tools function as "force multipliers"—by handling the low-level and tedious work, they allow humans to achieve more in the same time frame. All participants (P1–P8) described some form of productivity improvement, whether it was faster coding, quicker access to information (e.g., P3 obtaining instant code examples via ChatGPT instead of searching online), or reduced workload in writing and documentation (e.g., P5 drafting course materials with AI assistance).

5.1.2 Support for Documentation and Knowledge Sharing

Another prominent finding is the role of AI in enhancing documentation and communication within the team. Participants like P1, P5, and P6 demonstrated that generative AI serves as a writing assistant and knowledge repository.

P1, a product manager, used ChatGPT to "document sprint decisions" and draft meeting notes. This helped ensure that important discussions and decisions were recorded clearly

without consuming excessive time from team members. P5 similarly used AI to create structured technical documentation and even educational content, noting that the AI could produce a well-organized first draft of a design document that he would then lightly edit.

It saved time for the participants (who no longer had to start from a blank page), and it potentially improved the clarity of the documentation.

5.1.3 Trust and Reliance on AI

The findings also detail a cautious approach about trust, reliability, and accuracy of AI outputs. A recurrent theme was that developers do not fully trust AI-generated code without verification. All participants insisted on keeping a human in the loop—reviewing, testing, and validating the AI's suggestions before integrating them into the codebase.

For example, P7 recounted:

> "Even if it looks fine, I double-check every line"

of AI-written code, because sometimes the AI would produce unconventional solutions. P4 similarly stressed that one:

> "Can't just assume it's correct,"

and warned colleagues to review AI contributions as one would review a junior developer's work.

Participants built confidence in these tools gradually, using them first in low-risk scenarios (as noted in Chapter 4 and reiterated by P1 and P8 who started with non-critical tasks). Only after the AI proved helpful and mostly accurate in those scenarios than they extend its use to more significant coding tasks.

Participants encountered instances of the AI hallucinating—that is, producing outputs that looked plausible but were incorrect or irrelevant. P3's observations highlight this: vague prompts would yield random solutions, shows that the AI could easily go off-track if not guided properly. Some participants noted the AI's lack of insight into why a piece of code is needed or whether it truly fits the developer's intent.

A few participants (notably P3 and P7) expressed discomfort that AI-generated answers came without sources or explanations. Unlike a human colleague who might cite a textbook or a website, ChatGPT simply presents an answer confidently. This lack of traceability made some participants uneasy, as they had no direct way to verify an answer's origin or accuracy

except by testing it themselves. In a software team context, such opacity can be problematic for building trust in the tool.

A major takeaway is that trust remains a barrier—developers will not (and arguably should not) blindly accept AI output. Instead, they incorporate AI in a manner that maintains human oversight. In practice, this meant extra rounds of testing AI-written code, cross-checking AI-provided information, and sometimes using the AI as a second opinion rather than a primary authority.

5.1.4 Shifts in Collaboration and Team Dynamics

The integration of AI assistants brought notable shifts in how team members collaborated and distributed work. One observed theme was a trend toward greater individual autonomy in problem-solving. With tools like ChatGPT available at any time, developers—especially those at junior or mid-level—sometimes turned to the AI for quick guidance or solutions instead of immediately asking a senior colleague.

P3's experience illustrates this dynamic: rather than interrupting a teammate to ask for debugging help, he could describe the issue to ChatGPT and often get a useful explanation or hint. As a result, less experienced developers might rely slightly less on senior team members for routine questions.

The literature suggests that Copilot users felt less dependent on colleagues for help Marchesi et al., 2025, which resonates with our participants' anecdotes. On the flip side, this independence raised a question: are teams at risk of siloing when each member has a personal AI assistant?

P8, emphasized that he would not allow AI to work on critical code that needed collective understanding (such as core architecture), implicitly acknowledging that some tasks should remain collaborative human efforts.

Some participants used AI to brainstorm ideas in a group setting—P6 mentioned using ChatGPT to facilitate ideation during design sessions, almost like an unbiased participant that could suggest creative approaches on demand.

5.2 Alignment with Previous Research

The findings from our case study extend insights from prior research on generative AI in software development. Our results align with the growing body of evidence that AI coding assistants can boost developer productivity, while also echoing known challenges around

trust and correctness. In some areas, our study provides a new perspective – for example, on documentation practices and cross-role usage – that has been less emphasized in earlier work. We now compare our key findings with relevant literature and industry surveys, highlighting agreements and discrepancies.

5.2.1 Productivity Gains and Modes of AI Use

Our participants' experiences affirm what several controlled experiments and surveys have found: AI assistance can significantly accelerate certain programming tasks. For instance, a recent GitHub experiment reported that developers using Copilot completed a coding task approximately 55% faster than those without it (Ponsonby, 2024). This is directly in line with anecdotes from P2 and P4, who felt that coding with Copilot was markedly faster, especially for boilerplate portions of the code.

The subjective boost in efficiency we recorded participants feeling more productive and able to focus on important tasks—mirrors survey results where 88% of developers reported feeling more productive with AI tools (Ponsonby, 2024). These parallels lend credence to our qualitative findings, situating them in a wider context: we are observing, at the team level, the same productivity uplift that has been measured in broader populations.

Prior research by Barke et al. (2023) offers a conceptual lens through which to view these productivity improvements. Barke and colleagues found that programmers interact with code-generating AI in two primary modes: an "acceleration mode" (when working on a known problem and using AI to do it faster) and an "exploration mode" (when venturing into unfamiliar territory and using AI to get suggestions or learn) (Piller, Srour & Marion, 2024).

In our findings we saw acceleration mode in cases like P4's—he knew what needed to be done and used Copilot to implement it more quickly, essentially speeding up a straightforward task. Rxploration mode was also evident when participants like P3 and P6 used ChatGPT to brainstorm or to resolve doubts in new domains.

One nuance where our findings extend the literature is the impact on developer satisfaction. Several participants noted that offloading tedious tasks to AI made their work more enjoyable. This qualitative observation matches with GitHub's survey note that 74% of developers felt AI allowed them to focus on more satisfying work (Chang, 2023).

5.2.2 Documentation and Communication

Prior literature has not deeply focused on documentation, but some recent works hint at AI's potential in this area. Ebert and Louridas (Ebert & Louridas, 2023) explicitly note that LLM-based tools excel at tasks like "summarizing documentation, reviews, interviews, meeting minutes," thereby freeing up developers' time. Our findings strongly corroborate this point: participants like P1 and P5 achieved exactly that by using ChatGPT to summarize meetings and draft documentation much faster than they could have on their own. The AI's strength in natural language generation ensured that the documentation was not only quicker to produce but also more structured and clear.

Our observation that AI can help bridge understanding between technical and non-technical team members has being discussed in wider academic and industry discourse. (2023) (a pseudonym here for various industry reports) and others have discussed how AI tools embedded in IDEs or chats can act as on-demand "mentors" or explainers for developers looking to grasp unfamiliar code or concepts. Our participants extended this idea to the team level that AI explain code to developers and it can also explain code or technical plans to non-developers. This point isn't prominent in earlier academic research, which has focused mostly on coding tasks, but it aligns with a trend in industry where tools like ChatGPT are widely used by professionals in management, QA, etc., to comprehend technical topics.

There is also agreement between our participants' experiences and prior research regarding information retrieval efficiency. Developers often spend significant time searching for solutions or documentation (the classic Stack Overflow-driven workflow). AI assistants have begun to change that: a Microsoft developer survey in 2023 noted that 77% of developers felt they spent less time searching for information when using AI (Chang, 2023). Participants like P3 said that rather than doing multiple web searches for a bug or API usage, a well-phrased question to ChatGPT often yielded an immediate answer.

5.2.3 Reliability Concerns and Trust

The caution we observed among participants is strongly supported by prior studies. Many researchers have highlighted concerns regarding the correctness of AI-generated code and the continued necessity of human validation. For example, Ebert and Louridas (Ebert & Louridas, 2023) note that the more complex and detailed the task given to an AI, the higher the likelihood that the result will contain errors. This directly aligns with our participants' experiences: while AI was generally reliable for simple prompts (e.g., "generate a function to parse dates"), it often faltered when handling more complex requests (e.g., solving a tricky algorithm or managing multiple conditions simultaneously).

P7's line-by-line checking of AI outputs and P3's need to iteratively refine prompts reflect exactly what the literature warns: AI cannot be blindly trusted to get intricate logic correct.

Our findings shows that AI confidently giving an incorrect solution or a misleading snippet that participants had to catch. In alignment with these concerns, guidelines in both research and industry insist on human oversight. A Snyk report on AI code generation succinctly advises that developers "should always review AI-generated code… treating it as a suggestion rather than a final implementation" (Thoemmes, 2025)."

5.2.4 Impacts on Team Collaboration and Roles

The effects of AI on team dynamics are only beginning to be studied, but some early insights align with our observations. For example, Wivestad et al. (Marchesi et al., 2025) studied Copilot users in agile teams and found that participants reported focusing more on engaging tasks and feeling somewhat less dependent on teammates. Our participants similarly reported a sense of increased autonomy, with less need to seek help for trivial issues and a greater ability to focus on more complex or interesting design problems once AI handled the boilerplate work.

However, Wivestad et al. also noted that non-users in the same environment were cautious and sometimes skeptical, reflecting resistance or cultural friction around AI adoption. In our study, we did not have "non-user" participants to provide a counterpoint (all interviewees had at least tried the AI tools), but participants did relay that some colleagues outside the study or initially in the pilot were hesitant to use the AI, echoing that split in attitudes.

This suggests that within any given organization, the range of acceptance can be broad, a finding consistent with diffusion of innovation theory and supported by recent surveys. For instance, a GitHub survey (2023) found that 92% of developers have now tried AI tools; however, this still implies a minority who have not, often due to distrust or organizational policy (Chang, 2023). Our participants' experiences support the idea that while AI adoption is accelerating, resistance and caution still persist—often for valid reasons such as concerns about code quality or data security.

Where our study breaks some new ground relative to prior work is in highlighting cross-functional usage and its implications. The literature has thus far mostly centered on developers and occasionally on students learning programming. Our inclusion of roles like product managers, QA specialists, and designers using AI shows that the "AI in software development" conversation is broader than just coding.

We found that such usage can improve collaboration (e.g., clearer specs) and speed up

workflows beyond code, which suggests future research should examine these roles as well. Nonetheless, the principles observed still align with general findings: any role using AI will face the trust and correctness issues, and any role can potentially gain efficiency and creative support from AI. In that sense, our findings do not contradict existing research but rather extend it to a more holistic team context.

An interesting point of alignment comes from very recent industry survey data indicating that a large majority of developers expect AI to enhance team collaboration going forward (Chang, 2023).

5.3 Implications for Practice

As AI-assisted development moves from novelty to norm, teams and organizations will need to adjust their workflows, roles, and strategies to use these tools' potential while mitigating risks. Here we outline how different roles and team practices might evolve with the advent of generative AI, based on our findings and aligned with emerging industry trends.

5.3.1 Evolving Developer Roles and Skills

One clear implication is that the role of a software developer is gradually shifting from solely "code author" towards a combination of code curator and verifier. Given that AI can generate a significant portion of routine code, developers may write less boilerplate themselves; however, their responsibility in reviewing, testing, and refining AI-generated code becomes paramount.

Developers might spend more time on higher-level design and architectural thinking. Since AI handles micro-level coding tasks to an extent, developers can focus on system design. Team leads and architects might involve more developers in design reviews and architectural discussions, effectively upskilling the team's design capabilities.

The traditional skill set therefore broadens: familiarity with AI tools and understanding their limitations becomes as important as knowledge of a programming language's syntax. We foresee job descriptions beginning to include proficiencies like "experience with AI coding assistants" or "ability to validate AI-generated outputs" as desirable qualities.

Teams may also designate certain members as AI tool champions, individuals responsible for exploring new AI features or tools (such as Copilot's latest update or a new code-generation service) and sharing best practices with the rest of the team.

5.3.2 Changes in Team Workflow

Agile and DevOps practices will likely incorporate AI assistance at multiple points in the software lifecycle. For example, in agile sprint planning, a product owner or scrum master might use an AI tool to help refine user stories or acceptance criteria before presenting them to the team. This can make planning meetings more efficient, as the initial draft of requirements is clearer (as P5 experienced using AI for spec writing).

During development, code reviews may evolve as human reviewers focus more on complex logic, while simpler issues such as style violations or minor bugs are caught by AI analysis tools like DeepCode or Snyk, which are integrated into the pull request pipeline. Integrating AI-driven static analysis for security, style, and other concerns into CI/CD pipelines is a practical and impactful implication.

QA engineers might use generative tools to produce a broad range of test cases or even to generate test data, thereby speeding up the testing phase. The use of AI by our participant P2 to generate unit tests illustrates this: Imagine scaling that so that when a developer writes a new feature, an AI suggests a suite of unit tests. This can be integrated into development environments or CI systems (some early tools already attempt this). To apply this effectively, teams must set guidelines on what types of tests are trustworthy when generated and ensure human QA verifies critical scenarios.

Teams might establish norms for when to use AI versus when to consult a colleague. For example, an organization might encourage developers to attempt a solution with AI (for a limited time) and then, if unresolved, escalate to a human discussion.

5.3.3 Project Management and Planning

Project managers (PMs) and technical leads can also harness AI to improve planning accuracy and resource allocation. If AI accelerates the development of certain tasks, estimation practices might change. Teams could gather data on how much quicker they complete user stories with AI assistance and adjust their velocity calculations accordingly.

For example, a task that might have been estimated as a 5-point effort might effectively feel like 3 points with Copilot. Over time, this could reflect in sprint planning as the team consistently finishes more within the same timebox. PMs should be aware of this potential and revisit their estimation techniques, perhaps initially keeping some buffer for AI “hiccups” (such as time lost to debugging AI mistakes) until there is confidence in the net time saved.

5.3.4 Empowering QA and DevOps

Our findings suggest that AI tools are not limited to developers writing application code; they can also assist in QA, operations, and other support roles. QA engineers, for instance, can use generative AI to translate acceptance criteria into automated test scripts or to generate edge-case scenarios that a human might overlook. For example, given a description of a feature, an AI might propose unusual inputs or sequences for testing, thereby augmenting the tester's creativity.

Similarly, DevOps personnel can employ AI for tasks such as writing deployment scripts or server configuration templates. For instance, if an operations engineer needs to configure a CI pipeline or write an Infrastructure-as-Code (IaC) script, AI tools can accelerate the drafting of those configurations. Some participants in our broader study, including those in roles such as data scientists or designers, indirectly referenced similar applications when they used AI to script repetitive tasks.

5.4 Challenges in Real-World Adoption

Real-world adoption comes with a set of significant challenges. Our study and others highlight several constraints that organizations must contend with: trust and reliability issues, AI hallucinations, alignment of tools to tasks (tool fit), the need for new skills, and cultural or individual resistance. Here we discuss each of these challenges and their implications, drawing on participant experiences and broader observations.

5.4.1 Maintaining Trust in AI Outputs

Trust is a foundational challenge. Developers and managers are unlikely to fully embrace AI assistance if they cannot trust the outputs to be correct and safe. Time and effort must be invested in reviewing AI-generated results. If an organization observes that developers frequently have to re-do the work produced by AI, it may begin to question whether the tool is worthwhile. Building trust in AI systems is therefore critical for sustained adoption.

One hurdle is the opacity of AI decisions: modern large language models (LLMs) do not explain why they produce a particular piece of code or answer. This lack of transparency was problematic for participants who wanted assurance about the validity of AI-generated solutions. Real-world adoption may require the introduction of tools or processes that increase transparency, such as AI systems that provide source citations for their responses or, at a minimum, confidence scores. Since many coding assistants currently lack these features, trust must be built over time through consistent positive experience.

Gradual integration, as recommended in Section 5.5.1, is one way teams can allow trust to build organically. Another approach involves developing an internal validation suite. For example, after the AI generates code, comprehensive tests or static analyzers can be run automatically. Over time, if these checks rarely flag issues, developers are likely to gain confidence in the AI's reliability. Conversely, frequent detection of AI-generated mistakes may further erode trust.

5.4.2 Hallucinations and Reliability of AI

AI hallucinations, defined as outputs that are incorrect or irrelevant, pose both technical and operational challenges. From a technical standpoint, even state-of-the-art models occasionally produce subtly incorrect code (such as off-by-one errors or inefficient logic) or blatantly wrong answers (such as the misuse of a non-existent API). Participants in our study encountered these issues regularly, requiring manual debugging and correction. In a fast-paced development environment, such AI errors can be frustrating and potentially harmful if not identified promptly.

This challenge is acknowledged widely; researchers note that LLM performance can degrade on edge cases or when given ambiguous instructions (Thoemmes, 2025). The cost of hallucinations is not just the time required to fix them, but also the risk of false confidence: a developer might assume the AI-generated code is correct and fail to test it thoroughly, leading to issues down the line.

5.4.3 Tool Appropriateness and Integration (Tool Fit)

Not every AI tool fits every task or aligns with every team's workflow. One key challenge is selecting the right tool for the job. In our study, participants had licenses for specific tools (such as ChatGPT and Copilot), but they found that each tool offered distinct strengths and limitations. For example, ChatGPT—particularly the standalone web version—was well-suited for brainstorming and question-and-answer tasks but was not directly integrated into coding workflows. Copilot, on the other hand, was useful for in-IDE suggestions but sometimes lacked awareness of broader context, such as content from lengthy documentation. A specialized tool like DeepCode was valuable for static analysis but did not support code generation.

Startups and companies must determine when and where AI usage makes sense. For instance, early-stage prototyping might benefit hugely from AI (e.g., quickly trying out ideas), but later-stage hardening of a product (optimizing performance, security) might see less use of AI because the tasks are very specific or sensitive. This indicates that the "fit" of AI tools can vary across the development lifecycle.

5.4.4 Prompt Skill Requirements and Team Training

The rise of generative AI in software development has introduced a new and less conventional skill set: *prompt literacy*, or the ability to communicate effectively with AI tools. As reported by participants in this study, the quality of AI outputs was often closely tied to the clarity and structure of the prompts provided. For example, participants P3 and P7 noted that early attempts to use AI tools produced suboptimal results due to vague or poorly framed queries. Over time, they learned to construct more precise prompts, significantly improving the utility of the responses received.

Not all developers will intuitively master prompt crafting, and in some cases, poor use of AI tools may lead to inefficiencies, frustration, or erroneous outputs. Unlike traditional programming languages—where syntax and behavior are well-defined—the principles of prompt engineering are largely tacit and experience-based. This makes it difficult to design conventional training curricula around generative AI usage.

Some participants described emerging practices within their teams, such as informal mentorship, sharing effective prompts during stand-ups, or creating "prompt playbooks"—internal reference documents listing commonly used queries and successful prompt strategies. These resources aim to institutionalize knowledge around prompt engineering and reduce the learning curve for new users.

5.4.5 Resistance and Ethical Concerns

The human factor challenge lies in the reality that not everyone will welcome AI tools. Resistance may come from developers who fear that AI could deskill their roles or even replace them, or from those who simply distrust the quality of AI-generated output. This skepticism is especially common among veteran engineers who have witnessed the rise and fall of many overhyped tools. Resistance may also arise from management or clients who are concerned about issues such as intellectual property (IP) leakage, regulatory compliance, or the unpredictability of AI behavior.

In our findings, some participants noted colleagues who were reluctant to adopt AI tools, and one initial participant withdrew from the study after not finding immediate value. Overcoming resistance requires addressing both practical concerns and psychological comfort. On the practical side, transparent policies about AI use can help. For instance, organizations can clarify that any code generated by AI will be reviewed to maintain quality, and that AI is intended to assist rather than replace developer creativity or responsibility. On the cultural side, involving skeptics in pilot programs may ease their concerns, as it allows them to observe firsthand what AI can and cannot do. It is also helpful to share internal success stories.

Demonstrating, for example, how a teammate used AI to fix a bug in half the usual time can build confidence and generate positive peer influence that encourages adoption.

Several participants in our study (P4, P7, P8) were acutely aware of the risks associated with providing sensitive data to AI systems. They avoided inputting proprietary code or confidential information into AI tools. This concern is likely one reason why companies such as Samsung have banned or restricted the use of ChatGPT following incidents involving sensitive data leaks (Bloomberg News, 2023). To support real-world adoption, organizations will need to establish clear usage guidelines and consider technical safeguards, such as deploying on-premises AI models, to prevent unintentional disclosure of confidential information.

5.5 Recommendations for Startups

Table 6: Guidelines for AI Adoption in Startup Development Teams

Guideline / Best Practice	**Description**	**Supported By**
Start with Low-Risk Tasks	Use AI first for testing, formatting, or minor refactoring before scaling up	P1, P4, P8
Avoid Using AI with Sensitive or Proprietary Data	Refrain from pasting raw production code or user data into public AI tools	P4, P5, P7, P8
Develop Prompt Literacy Across the Team	Train team members on how to write clear, contextual prompts	P3, P5, P7
Use AI to Support, Not Replace, Critical Thinking	Always review and test AI-generated outputs before implementation	All participants
Empower Non-Technical Roles with AI	Enable non-dev staff to use AI for specs, documentation, and planning	P5, P6, P7
Define Clear Boundaries for AI Use	Decide in advance what AI should and shouldn't be used for	P2, P4, P8
Maintain Human Oversight on Core Logic and Architecture	Keep key business logic and architectural decisions human-led	P1, P2, P4

Based on our findings and the discussed challenges, we propose a some recommendations for software development teams, especially startups and smaller organizations, to use generative AI effectively. Startups often operate with limited resources and high uncertainty, so the way they adopt new tools like AI can significantly impact their productivity and product quality.

Each recommendation below is grounded in our interview findings, reflecting participants' experiences and practices, and is further supported by insights from relevant literature or in-

dustry examples where applicable. We present seven key recommendations, each addressing a critical aspect of AI integration. These range from initial adoption strategies to considerations around data privacy, team capabilities, and the importance of maintaining robust engineering practices.

5.5.1 Start with Low-Risk Tasks

When introducing AI into the development workflow, begin by applying it to low-risk, non-critical tasks. This phased approach allows the team to experiment and learn the tool's capabilities without endangering core project quality and sensitive data. Our participants repeatedly echoed this strategy. P1 described that their team's initial use of AI was for "things like note-taking and documenting sprint decisions, things that wouldn't break production." By doing so, the team could explore how the AI behaves, what kind of outputs it produces, and how much they could trust it—all in a sandbox of sorts that did not impact live features or crucial algorithms.

P4 similarly tested AI in minor bug-fixing and code clean-up tasks at first. He recalled, "Initially, I just used it for suggestions and minor fixes" for example, asking the AI to propose a refactoring of a small function or to suggest a better naming for variables. Only after these small-scale successes and gaining confidence did he escalate AI's role to larger code writing tasks (eventually using it to help implement "whole features" as noted earlier).

From an industry perspective, this recommendation aligns with general change management best practices: pilot new technology in a limited scope, evaluate, then scale up. It also matches with how other companies have approached AI; anecdotal reports suggest many organizations first adopted Copilot for writing unit tests or internal tools before using it in production code.

A startup implementing this could identify a handful of low-risk use cases—for example: AI-generated code comments, conversion of user stories into task lists, test-case generation for non-critical modules, formatting code to style guidelines, or producing draft emails/responses for routine customer queries. Team members use AI in these areas, review the outcomes, and share feedback on what worked or needed fixing. Only after several iterations (sprints) of satisfactory performance would they consider moving AI "upstream" into core development tasks.

This approach was strongly supported by our participants' experiences (P1, P4, P8 all underscored it) and is our top recommendation for any team new to generative AI.

5.5.2 Avoid Using AI with Sensitive or Proprietary Data

One of the most important policies for organizations is to never expose confidential code or data to public AI services. Our study participants were keenly aware of this risk, and we strongly recommend formalizing this caution into team guidelines. Generative AI tools like ChatGPT and Copilot are often hosted on external servers; any input to these tools could potentially be seen by the service provider or used to train future models. Therefore, proprietary source code, customer data, passwords, or any sensitive information should not be directly input into AI prompts.

P4 emphasized this practice, stating, "I try not to paste company-specific code into it. I usually describe the structure instead, just to be safe." In other words, if he needed AI help, he would abstract the problem: for example, instead of pasting a proprietary algorithm, he might say "I have a function that does X, but it's running slow" and seek generic advice. P7 similarly mentioned that he "never paste[s] our actual repo code" and instead describes issues in abstract terms.

P5 shared that his company had proactively created internal guidelines warning educators "not to paste any textbook content or full lesson plans into AI, because they could be copied later by someone else." This underscores a realistic threat: any information entered into a public AI system could potentially appear in another user's output. There have been notable incidents illustrating this risk. For instance, Samsung experienced a case in which engineers entered proprietary source code into ChatGPT, prompting the company to implement a ban on such tools (Bloomberg News, 2023).

Startups, which often handle proprietary or sensitive information, should be especially vigilant when using AI tools. This recommendation can be implemented in practice by establishing a clear policy and ensuring it is communicated to all team members. The policy should outline specific categories of information that are prohibited from being included in AI prompts, such as code from private repositories, personal data, or unreleased product details. It is also advisable to provide concrete examples and, if possible, require developers to complete a brief training or formally acknowledge the policy.

5.5.3 Develop Prompt Literacy Across the Team

To get the most out of AI tools, it is essential that team members become skilled in writing prompts. "Prompt literacy" refers to the ability to communicate with generative AI in a precise, context-rich manner that yields useful and accurate responses. Our research indicates that without this skill, users may receive irrelevant or incorrect outputs, leading to frustration or misuse of the AI. Therefore, we recommend that startups invest time in training the

team on how to write good prompts, sharing tips and tricks, and encouraging a culture of continuous learning in this area.

Participants in our study realized the importance of prompt quality through experience. P3 noted that when he provided specific detail to the AI about the code in question, the responses were much more helpful: "When I give the exact function or the exact part of the code, it works well… but if I just say 'fix this', it gives me something random." This underscores a common pattern: vague questions result in poor answers, whereas detailed, well-scoped questions yield better outcomes. P7 similarly shared that he learned to guide the AI step-by-step: "At first, I used to type long questions and get bad results. Then I realized I have to guide it—step by step."

The need for prompt literacy extends beyond developers to include any team member who interacts with AI tools. P5 mentioned that in his context, "We train everyone to prompt correctly, not just developers. Even marketing and sales use it now." This indicates that a formal or informal training effort was underway, emphasizing that effective prompting is not necessarily intuitive but can be learned through guided instruction.

For a startup, developing prompt literacy can take several forms. One is to hold internal workshops or brownbag sessions where team members demonstrate how they use the AI. They could show examples of a poor prompt versus a well-crafted prompt and the difference in AI outputs. Another approach is to create an internal knowledge base or cheat sheet of prompting techniques that work well for the team's specific tools and tasks. For example, a shared document might include patterns like: "If you want to debug code, first ask the AI to explain the code snippet in plain language, then ask about the bug," or "When asking for code, specify the language and version, and include any error messages."

5.5.4 Use AI to Support, Not Replace, Critical Thinking

Generative AI should be viewed as a support tool rather than a replacement for human judgment and critical thinking. This principle is crucial to ensure that software quality and team knowledge do not degrade over time. We recommend that teams establish a norm: always review AI outputs, and treat AI suggestions as if they came from a junior developer—helpful, often correct, but requiring oversight.

All participants in our study concurred with this approach. They consistently cautioned against blind acceptance of AI-generated code or answers. P4 insisted that "Developers must review AI code before using it. You can't just assume it's correct." Similarly, P8 warned about the risk to "stop thinking and just copy"—if developers do that, "you lose the habit of problem-solving." These insights highlight that while AI can deliver quick answers,

the team's expertise remains paramount in validating those answers.

From a best practice standpoint, this means that for any code written by AI, a human developer should: inspect it for logical correctness, adapt it to the codebase's style and context, run tests against it, and consider edge cases the AI might have missed. Teams might formalize this by including in their Definition of Done that "any AI-generated code must undergo the same or stricter review as human-written code."

For non-code outputs like documentation or plans that AI helped draft, similar rules apply: always proofread, correct factual errors, and adjust tone/style as needed. Another facet of this recommendation is avoiding over-reliance on AI for decision-making. AI can suggest what code to write or how to fix a bug, but decisions like what architecture to choose, what features to prioritize, or how to resolve a design trade-off are best left to human deliberation.

As P1 (from a product management perspective) noted, he might brainstorm feature ideas with ChatGPT, but "the architecture still comes from the team." Teams can explicitly mark certain areas as "AI-free zones"—e.g., core system design, security-critical decisions—where only human expertise is applied. This ensures that critical thinking is exercised where it matters most.

Junior developers should be encouraged to understand why a certain AI-suggested code works—not just copy-paste it. They could be asked to explain to a senior (or in a comment) the rationale of the AI-generated code they incorporated, as if it were their own. If they can't, that's a red flag to reevaluate the code.

Supporting critical thinking also ties into error handling: if the AI gives an answer that seems off, developers should question it, debug it, or ask the AI to clarify. We saw participants not hesitating to double-check (P7 double-checked every line, P3 iteratively questioned the AI). This practice should be encouraged and normalized—using AI doesn't mean taking shortcuts on due diligence.

In fact, research and industry voices echo this recommendation strongly. A Snyk report on AI in coding explicitly advises developers to treat AI output as a suggestion requiring validation and testing(Bloomberg News, 2023). Many blog posts and early adopters share the rule "always review AI code as if written by an intern" (Keployio, 2023).

5.5.5 Empower Non-Technical Roles with AI

Generative AI tools can also be transformative for non-technical roles in a software team, such as product managers, designers, quality assurance specialists, technical writers, or even

marketing and customer support staff who interface with the development process. We recommend consciously extending AI access and training to these roles, so that they can independently accomplish tasks that previously required heavy developer involvement.

Our study provided clear examples of this. P5, who works in an educational technology context, used AI to help write structured development specifications and even to assist teachers in drafting content. He noted, "I can now write structured dev specs thanks to AI," illustrating that someone not writing code day-to-day can still benefit from AI assistance in a technical domain. P6 shared that product owners or project managers in his team found value in AI-generated documentation: "It's easier for the product owner or the PM to read AI-generated documentation—it's clearer. P7 observed a broader organizational trend: "Even people from marketing or customer support use AI to write drafts or understand technical stuff."

For a startup, empowering non-technical team members with AI can yield several benefits:

- **Faster turnaround** on tasks like writing requirements, generating reports, summarizing meetings, or drafting test plans, because these team members can use AI to produce initial versions and then refine them without waiting on an engineer's help.
- **Improved communication** between technical and non-technical staff, as AI can act as an intermediary to explain or rephrase information.
- **Greater inclusion and innovation**, as more team members can contribute to traditionally technical activities—like brainstorming solutions or reviewing documents—thereby enriching the development process with diverse perspectives.

To implement this, startups should provide AI tool access and training to all roles that intersect with the development process. This might mean acquiring additional licenses (e.g., ChatGPT Plus or GitHub Copilot) for non-engineers. It also requires showcasing how AI might assist each role: a product manager could draft requirement documents or synthesize user feedback; a QA specialist might use AI to generate edge-case test scenarios from user stories.

Importantly, just as with developers, non-technical roles should also be trained on the limitations of AI. They must understand that AI might fabricate plausible-sounding but incorrect information, and should therefore validate outputs or seek clarification when in doubt.

5.5.6 Define Clear Boundaries for AI Use

It is important for teams to deliberately decide what AI should and shouldn't be used for within their workflows. By defining clear boundaries. AI tools should be applied in areas of strength and avoided in contexts where they might cause more harm than good.

Our participants practiced this, and we suggest making it an explicit policy. P2 mentioned an example boundary: in their usage, "For the testing we used it—for generating test cases, and test summaries, test reports. For such things it was very good." This implies they limited AI use to testing documentation and generation, which are areas where an error would not be catastrophic and where AI's ability to enumerate scenarios was beneficial.

P4 provided another perspective on boundaries: "We try to give it limited tasks—small functions, or parts of the code where we know what the output should be." His team intentionally constrained AI to solving well-defined sub-problems where the result could be easily validated. They did not let AI design the overall module architecture or handle data-sensitive encryption logic.

To implement clear boundaries, a startup should hold a team discussion (e.g., during a retrospective or tooling review) to outline categories of tasks for AI. For example, they might conclude:

- **AI is recommended for:** code formatting, boilerplate CRUD generation, unit test scaffolding, translating comments, documentation drafting, and non-critical algorithm suggestions.
- **AI is discouraged or forbidden for:** writing cryptographic code, handling user data transformations, generating core business logic, or tasks tied to legal compliance without expert oversight.

Boundaries can also be tied to review levels: for example, AI-generated design diagrams may be used but must be reviewed in a team meeting before implementation.

Boundaries can be role-specific. QA might use AI freely for test generation, while developers may be restricted from using AI for production hotfixes without review. PMs might rephrase requirements with AI but not define final acceptance criteria without engineering input.

Defining clear boundaries also helps ease skepticism. Literature on AI adoption emphasizes the importance of balanced and mindful integration, which aligns with our recommendation to determine where AI complements team objectives rather than compromises them (Marchesi et al., 2025).

5.5.7 Maintain Human Oversight on Core Logic and Architecture

No matter how capable AI tools become, it is crucial to keep human experts in charge of core system logic and architectural decisions. This recommendation is about preserving the integrity and coherence of a software system.

Our findings suggest that final authority and understanding must remain with the development team, particularly when it comes to the ”big rocks“ of a project—such as key algorithms, system structure, and inter-component contracts. Participants consistently emphasized this point. P1, speaking from a product management perspective, explicitly stated that even though he used ChatGPT to brainstorm and outline ideas, ”the architecture still comes from the team.“ This establishes a clear boundary: while AI can assist with creative ideation, the responsibility for architectural design remains with humans.

Architecture involves trade-offs, nuanced requirements, long-term maintainability, and alignment with business strategy. These are areas where human experience and holistic understanding far exceed what current AI systems can offer. P4, a senior developer, also emphasized the importance of human oversight, stating, ”You should definitely review the AI code instead of just pasting [it],“ and applied the same logic particularly to critical architectural elements.

Even when AI assists in generating architectural outputs (e.g., diagrams or lists of services), humans must review and revise them. Teams may adopt policies such as:

- All architectural proposals must be reviewed in an engineering meeting.
- Core modules require manual code review by senior developers, regardless of perceived simplicity.

Human oversight also ensures long-term maintainability. If AI writes large portions of core logic without human understanding, it creates “black box” risks. Teams should ensure anyone integrating AI-generated core code understands it well enough to explain, debug, and maintain it. Humans must “own” the code—AI can help write, but not replace comprehension and accountability.

To operationalize this, startups can:

- Define what constitutes “core logic” and require sign-off by designated leads for changes to those areas.
- Use repository permissions to enforce code ownership and reviews.
- Restrict AI use in sensitive modules or increase scrutiny of AI-generated contributions.

5.6 Limitations and Suggestions for Future Research

This study has several methodological and practical limitations that must be acknowledged. First, the sample size and composition impose constraints on generalizability. The partici-

pants were relatively few in number and drawn from a specific professional and geographic context; as a result, the findings may not represent the full diversity of software development teams worldwide. Such limited sampling is a common issue in emerging AI-in-software-team studies – indeed, a recent review found that over 60% of related studies reported small sample sizes or region-specific samples as limitations, underscoring risks to broader representativeness (Ferino et al., 2025).

Second, the study relied heavily on self-reported data (e.g. interviews), which introduces potential bias. Participants' statements about productivity gains or challenges are subjective and may be influenced by optimism, novelty effects, or social desirability. Prior works have noted that self-reported assessments can diverge from actual behavior or performance (Ferino et al., 2025). For example, respondents might overstate positive outcomes or fail to accurately recall difficulties, as has been cautioned in similar GenAI adoption studies (Chan & Hu, 2023).

Third, the research was cross-sectional, capturing a snapshot in time shortly after the introduction of generative AI tools into workflows. It did not observe changes over an extended period. This is significant because generative AI for software development is in an early, rapidly evolving stage. The capabilities of tools (and developers' proficiency with them) are advancing quickly; therefore, findings at one point in 2024/2025 may become outdated as tools improve or users adapt. Scholars have explicitly warned that the fast evolution of LLM tools threatens to render results obsolete if studies do not keep pace (Ferino et al., 2025).

Another limitation is the scope of roles and tasks considered. This thesis concentrated on software developers and their coding-related tasks when using generative AI assistants. However, software development teams are often cross-functional, including roles such as QA engineers, UX designers, technical writers, and project managers. These roles might also leverage generative AI (for example, in generating test cases, UI prototypes, documentation, or user stories), but our study did not specifically investigate those perspectives. This narrow role focus means the interplay of AI assistance across different functions in a team remains unexamined here. (Chan & Hu, 2023) (Ferino et al., 2025).

Acknowledging these limitations opens clear avenues for future research. First and foremost, future studies should endeavor to include larger and more diverse samples of AI-assisted development teams. Broadening the participant pool across different organization sizes, industries, and regions will enhance generalizability (Chan & Hu, 2023) (Ferino et al., 2025). International or multi-site studies could capture cultural and organizational differences in how AI is adopted.

Second, researchers should consider longitudinal designs – studying teams over an extended period as they integrate generative AI into their workflow. A longitudinal approach would help observe how the impact of AI tools evolves (e.g. do productivity gains grow or plateau? does initial skepticism lessen over time?) and would mitigate the "snapshot" problem (Chan & Hu, 2023) (Ferino et al., 2025). For example, Banh et al. (2025) encourage long-term studies to track the maturation of GenAI services and organizational readiness over time (Banh, Holldack & Strobel, 2025).

Generative AI tools are improving (with new versions like GPT-4, code-specific models, etc.), and organizations are continuously adapting their practices. The research community should continually revisit earlier findings to see if earlier challenges (such as certain types of errors or team frictions) have been mitigated by newer tool features or greater user proficiency. (Ferino et al., 2025) (Chan & Hu, 2023)

6 Conclusion

6.1 Summary of Key Findings

Generative AI tools can substantially enhance developer productivity and efficiency in software teams. Participants in this study consistently reported time savings, faster completion of routine coding tasks, and relief from tedious programming work when using AI assistants. These qualitative insights align with emerging industry data – for example, analysts have estimated that integrating code-generation AI can boost development performance by 20–50% in the near term (Oracle Corporation, 2024).

AI assistance contributes to improved code quality and developer learning, but its benefits are best realized with human oversight. Team members noted that tools like ChatGPT or Copilot often catch errors (such as syntax mistakes or forgotten edge cases) and can suggest improvements or alternative approaches, which in some cases led to cleaner, more robust code. There is evidence that using AI in tasks like code review and documentation can streamline quality assurance – for instance, one study found that pull-requests documented with AI had higher approval rates and significantly faster review times than those written manually (Sifi, 2025). Similarly, developers found AI useful as a learning aid, quickly providing examples or explanations (akin to an on-demand mentor). However, the need for vigilant human review was a universal theme. Our findings echo widely reported concerns that AI-generated code may contain inaccuracies, insecure implementations, or unanticipated bugs (Banh, Holldack & Strobel, 2025).

Several challenges temper the benefits of generative AI, including issues of trust, correctness, and data privacy. One prominent challenge identified was AI output correctness – respondents encountered instances of AI "hallucinations" (plausible-looking but incorrect code) and contextually irrelevant suggestions. Another challenge revolves around privacy and security: using cloud-based AI tools raised questions about sensitive code or data being inadvertently exposed. Teams worried about proprietary code snippets included in prompts and whether that could breach confidentiality or licensing terms. This finding matches with the broader industry's caution that data privacy, intellectual property, and compliance must be managed proactively when deploying generative AI (Banh, Holldack & Strobel, 2025).

Human expertise remains central and irreplaceable, despite advances in generative AI. A major finding of this thesis is that AI is a powerful augmentative tool – not a wholesale substitute for developer expertise. Even as AI systems become more capable, there are clear limits to what they can autonomously achieve in realistic software projects. Our participants provided examples where human insight was required to make final decisions, understand

complex domain-specific requirements, or debug issues that AI could not resolve. This observation is backed by empirical evaluations in the literature: for instance, a large-scale experiment found that state-of-the-art code generation models could independently solve only about 4% of typical programming tasks when tested against a suite of real-world coding problems (Stokel-Walker, 2024).

6.2 Contributions to Practice

This research offers several concrete contributions to software development practice, particularly valuable for software startups and engineering team leads seeking to navigate the advent of generative AI tools.The thesis provides a set of evidence-based guidelines and best practices for integrating AI into development workflows. By documenting the experiences of early adopters, it highlights what works and what pitfalls to avoid. For example, teams are advised to start with pilot implementations of AI assistants on limited projects or subsets of the team to evaluate impact before scaling up. (Russo, 2024).

Our findings suggest that such pilot programs, followed by structured feedback sessions, allow teams to tailor the tool's use to their specific context (e.g. identifying that AI is particularly useful for generating unit tests, but perhaps less reliable for architectural decisions). We also emphasize the importance of thorough tool evaluation and selection: organizations should assess different AI coding assistants for how well they fit into existing toolchains and processes (Russo, 2024).

Another practical contribution is in outlining how team workflows and norms can be adapted to maximize benefits from AI. The research identified best practices such as: incorporating AI-generated code into the standard code review process (rather than accepting it unchecked), establishing coding standards and conventions that also apply to AI (for example, if an organization has a specific coding style or security guidelines, these need to be enforced on AI contributions too), and creating channels for developers to share prompt techniques and AI usage tips with each other. The thesis also highlights the role of organizational support and training in successful AI adoption. Engineering managers and team leads can draw on these insights to shape intervention programs – for example, scheduling training sessions or workshops for developers to learn how to effectively use prompts, interpret AI suggestions, and handle common failure cases. In line with recent frameworks (Russo, 2024).

For software startups, which often operate under tight resource constraints and aggressive timelines, this thesis offers guidance on leveraging generative AI to extend their capabilities. The findings reassure practitioners that AI coding assistants can accelerate develop-

ment—particularly for routine tasks and rapid prototyping—and reduce the burden on small teams. At the same time, we offer cautionary guidance to help prevent misuse. For example, startups handling sensitive client data are encouraged to implement governance policies that clearly define what can and cannot be submitted to an AI service in order to avoid data leaks (Kyndryl Corporation, 2025).

6.3 Final Thoughts

Generative AI is poised to become a staple in the software engineer's toolkit, much like compilers or version control – a technology that, when used correctly, fades into the background of everyday work while providing considerable productivity gains. In the coming years, we can expect AI to handle an expanding scope of developer tasks: not only code scaffolding and autocompletion, but potentially more complex activities like automated code refactoring, test generation, or even preliminary architecture drafting. Such developments could accelerate software production and innovation to an unprecedented degree, enabling teams to deliver more value in less time.

However, realizing this positive future depends on how responsibly and thoughtfully the software industry navigates the ethical and practical challenges identified. One key consideration is the human element: it bears repeating that developers are not being removed from the loop, but rather elevated into new roles. The fear that "AI will replace programmers", while natural in periods of technological upheaval, appears overstated given the current capabilities of AI (Stokel-Walker, 2024).

As discussed, contemporary LLMs cannot yet solve the majority of real-world coding problems independently, and it is unlikely in the near term that AI will operate without human guidance in complex projects. What is far more likely is a redefinition of developer roles – routine coding might become less central, while higher-level oversight, critical thinking, and decision-making become even more important. It will be crucial to manage this transition in the workforce. Companies and educational institutions should prepare developers for a world where working alongside AI is the norm. This includes training in how to effectively collaborate with AI (e.g. crafting good prompts, validating AI output, understanding AI's limitations), as well as instilling an ethical mindset for AI use. Notably, nearly half of developers in a late-2023 survey reported anxiety about AI threatening their jobs (Stokel-Walker, 2024)

The rise of generative AI is a transformative moment for software development teams – one filled with excitement for its possibilities and caution for its pitfalls. This thesis has argued that with informed strategies and human-centric approaches, generative AI can be

harnessed to augment human creativity and productivity rather than replace it. The final takeaway is one of balanced optimism: if we continue to study, refine, and govern the use of AI in our development processes, we can anticipate a future where software teams are more empowered and efficient, tackling ambitious projects that might have been out of reach before.

References

Alenezi, M. & Akour, M. (Jan. 2025). AI-Driven Innovations in Software Engineering: A Review of Current Practices and Future Directions. *Applied Sciences* 15(3). Open Access, CC BY 4.0, p. 1344. DOI: 10.3390/app15031344. URL: https://doi.org/10.3390/app15031344.

Banh, L., Holldack, F. & Strobel, G. (2025). Copiloting the Future: How Generative AI Transforms Software Engineering. *Information and Software Technology* 183, p. 107751. DOI: 10.1016/j.infsof.2025.107751. URL: https://doi.org/10.1016/j.infsof.2025.107751.

Bloomberg News (May 2023). Samsung Bans ChatGPT and Other Generative AI Use by Staff After Leak. URL: https://www.bloomberg.com/news/articles/2023-05-02/samsung-bans-chatgpt-and-other-generative-ai-use-by-staff-after-leak.

Capgemini Research Institute (July 2024). *Turbocharging Software with Generative AI: How Organizations Can Realize the Full Potential of Generative AI for Software Engineering*. Tech. rep. Accessed: 2025-05-21. Capgemini Research Institute. URL: https://www.capgemini.com/news/press-releases/generative-ai-is-set-to-be-adopted-by-85-of-the-software-workforce-over-the-next-two-years/.

Chan, C. K. Y. & Hu, W. (2023). Students' voices on generative AI: perceptions, benefits, and challenges in higher education. *International Journal of Educational Technology in Higher Education* 20(43). Open Access, CC BY 4.0. DOI: 10.1186/s41239-023-00411-8. URL: https://educationaltechnologyjournal.springeropen.com/articles/10.1186/s41239-023-00411-8.

Chang, S. (2023). *Prompt Engineering Examples for Software Development*. Accessed: 2025-05-25. URL: https://gist.github.com/sausheong/2f4c4845d44b31075bb1a3285ed40919.

Coutinho, M., Marques, L., Santos, A., Dahia, M., França, C. & Souza Santos, R. de (2024). The Role of Generative AI in Software Development Productivity: A Pilot Case Study. In: *Proceedings of the 1st ACM International Conference on AI-Powered Software (AI-ware '24)*. Porto de Galinhas, Brazil: ACM, pp. 1–8. DOI: 10.1145/3664646.3664773. URL: https://arxiv.org/abs/2406.00560.

Daigle, K. & Staff, G. (Aug. 2024). *Survey: The AI wave continues to grow on software development teams*. Accessed: 2025-05-21. URL: https://github.blog/news-insights/research/survey-ai-wave-grows/.

Ebert, C. & Louridas, P. (July 2023). Generative AI for Software Practitioners. *IEEE Software* 40(4), pp. 30–38. DOI: 10.1109/MS.2023.3265877. URL: https://doi.org/10.1109/MS.2023.3265877.

Ferino, S., Hoda, R., Grundy, J. & Treude, C. (2025). Junior Software Developers' Perspectives on Adopting LLMs for Software Engineering: a Systematic Literature Review.

arXiv preprint arXiv:2503.07556. Accessed: 2025-05-26. URL: https://arxiv.org/abs/2503.07556.

Gupta, A., David, J. S., Golli, A. & Pasupuleti, V. S. M. (2025). Generative AI in the Software Development Lifecycle (SDLC). *Global Journal of Engineering and Technology (GJET)* 3(1). ISSN: 2583-3359 (Online), pp. 26–36. URL: https://gsarpublishers.com/wp-content/uploads/2025/02/GJET312025-Gelary-script.pdf.

Keployio (May 2023). AI Code Generation: Revolutionizing Software Development. *Medium*. URL: https://medium.com/@keployio/ai-code-generation-revolutionizing-software-development-d76b284ab05c.

Kyndryl Corporation (Mar. 2025). *5 Lessons Learned Equipping Software Developers with Generative AI*. Accessed: 2025-05-26. URL: https://www.kyndryl.com/us/en/perspectives/articles/2025/03/gen-ai-for-app-developers.

Madupati, B. (July 2024). AI's Impact on Traditional Software Development. *Journal of Artificial Intelligence & Cloud Computing* 3(4). Review Article, Open Access, pp. 1–7. ISSN: 2754-6659. URL: https://www.onlinescientificresearch.com/articles/ais-impact-on-traditional-software-development.pdf.

Marchesi, L., Goldman, A., Lunesu, M. I., Przybyłek, A., Aguiar, A., Morgan, L., Wang, X., Pinna, A. & França, C., eds. (2025). *Agile Processes in Software Engineering and Extreme Programming – Workshops: XP 2024 Workshops, Bozen-Bolzano, Italy, June 4–7, 2024, Revised Selected Papers*. Vol. 512. Lecture Notes in Business Information Processing. Open Access, CC BY 4.0. Cham, Switzerland: Springer. ISBN: 978-3-031-72780-1. DOI: 10.1007/978-3-031-72781-8. URL: https://doi.org/10.1007/978-3-031-72781-8.

Nowell, L. S., Norris, J. M., White, D. E. & Moules, N. J. (2017). Thematic Analysis: Striving to Meet the Trustworthiness Criteria. *International Journal of Qualitative Methods* 16(1). Open Access, SAGE Journals, pp. 1–13. DOI: 10.1177/1609406917733847. URL: https://doi.org/10.1177/1609406917733847.

O'Flaherty, K. (Jan. 2025). *AI-generated code risks: What CISOs need to know*. Accessed: 2025-05-22. URL: https://www.itpro.com/technology/artificial-intelligence/ai-generated-code-risks-what-cisos-need-to-know.

Oracle Corporation (2024). *7 Ways GenAI Can Help Improve Software Development*. Accessed: 2025-05-26. URL: https://www.oracle.com/artificial-intelligence/generative-ai/generative-ai-software-development/.

Piller, F. T., Srour, M. & Marion, T. J. (Sept. 2024). Generative AI, Innovation, and Trust. *The Journal of Applied Behavioral Science* 60(4). First published online September 30, 2024. Open access. DOI: 10.1177/00218863241285033. URL: https://doi.org/10.1177/00218863241285033.

Ponsonby, C. (Jan. 2024). *Best of 2023: Measuring GitHub Copilot's Impact on Engineering Productivity*. Accessed: 2025-05-25. URL: https://devops.com/measuring-github-copilots-impact-on-engineering-productivity/.

QDAcity (n.d.). *Trustworthiness in Qualitative Research: Guba and Lincoln's Criteria*. Accessed: 2025-05-24. URL: https://qdacity.com/trustworthiness.

Russo, D. (2024). Navigating the Complexity of Generative AI Adoption in Software Engineering. *ACM Transactions on Software Engineering and Methodology* 33(5), Article 135. DOI: 10.1145/3652154. URL: https://doi.org/10.1145/3652154.

Sifi, A. (Mar. 2025). *How does generative AI impact Developer Experience?* Accessed: 2025-05-26. URL: https://devblogs.microsoft.com/premier-developer/how-does-generative-ai-impact-developer-experience/.

Stahl, N. A. & King, J. R. (2021). Expanding Approaches for Research: Understanding and Using Trustworthiness in Qualitative Research. *Journal of Literacy Innovation* 7(1). ERIC Number: EJ1320570, Accessed: 2025-05-24, pp. 60–77. URL: https://files.eric.ed.gov/fulltext/EJ1320570.pdf.

Stokel-Walker, C. (Jan. 2024). *Researchers say generative AI isn't replacing devs any time soon*. Accessed: 2025-05-26. URL: https://leaddev.com/career-development/researchers-say-generative-ai-isnt-replacing-devs-any-time-soon-2.

Thoemmes, S. (Mar. 2025). *AI Code Generation: Code Security & Quality, Benefits, Risks & Top Tools*. Accessed: 2025-05-26. URL: https://snyk.io/blog/ai-code-generation-code-security-quality-benefits-risks-top-tools/.

Wolpers, S. (Mar. 2025). *Generative AI in Agile: A Strategic Career Decision*. Accessed: 2025-05-21. URL: https://age-of-product.com/generative-ai-agile/.

Appendix A:Interview Questions for Software Developers

Warm-up Questions (5 minutes)

1. Where do you work, and what is your role?
   Can you describe your key responsibilities in the development process?
2. What roles have you held in software development?
3. What programming languages, frameworks, or tools do you work with most often?
   Do you specialize in a particular tech stack?

Section 1: AI Integration in Software Development (5 minutes)

1. How do AI tools fit into your software development workflow?
2. Can you describe the AI tools your team uses most frequently?

Section 2: Challenges & Risks of AI in Development (5–6 minutes)

1. What are the biggest challenges you have faced when using AI tools?
   Have you encountered technical, adoption, or integration issues?
   Have you noticed security risks, biases, or intellectual property concerns in AI-generated code?
   Are there any ethical concerns or skill gaps in AI-assisted development?

Section 3: AI's Impact on Productivity & Efficiency (5 minutes)

1. Has AI improved your efficiency as a product manager? In what ways?
2. Can you share an example where AI enhanced your productivity or decision-making?

Section 4: AI's Effect on Team Roles & Collaboration (5 minutes)

1. How has AI influenced your role or responsibilities in software development?
2. Has AI influenced how developers collaborate or communicate within teams?

Section 5: Best Practices & Recommendations for AI Adoption (4–5 minutes)

1. What best practices have helped your team successfully integrate AI into the development workflow?
2. If you could give one piece of advice to startups adopting AI-assisted development, what would it be?

Section 6: Overall Perception of AI in Development (5 minutes)

1. Overall, do you believe AI has benefited your development process? Why or why not?
2. What are the most important advantages you've personally experienced with AI-assisted development?
3. Do you think Generative AI has benefited junior or senior developers more?